

@azdocuments

INTRODUCTION TO DBMS

SUBJECT CODE: BCS755A

MODULE-5

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

INTRODUCTION TO DBMS

BCS755A

MODULE-5

SQL: Advanced Queries: More complex SQL retrieval queries, Specifying constraints as assertions and action triggers, Views in SQL.

Concurrency Control in Databases: Two-phase locking techniques for Concurrency control, Concurrency control based on Timestamp ordering, Multiversion Concurrency control techniques, Validation Concurrency control techniques, Granularity of Data items and Multiple Granularity Locking.

Textbook 1: Ch 7.1 to 7.3, 21.1 to 21.5

5.1 More Complex SQL Retrieval Queries

Introduction

This chapter covers these advanced features:

1. NULL comparisons and three-valued logic
2. Nested queries, tuples, and set/multiset comparisons
3. Correlated nested queries
4. EXISTS and UNIQUE functions
5. Explicit sets and renaming
6. Joined tables and outer joins
7. Aggregate functions
8. Grouping — GROUP BY and HAVING
9. WITH and CASE constructs
10. Recursive queries
11. Overall summary and query structure

5.1.1 Comparisons Involving NULL and Three-Valued Logic

The Three Meanings of NULL

NULL represents a **missing value**, but it can carry **one of three different interpretations**:

Interpretation	Meaning	Example
1. Value unknown	The value **exists** but is **not known**, or it is unknown whether it exists at all	A person's Date_of_birth is not known; or a person's Home_phone — we don't even know if they have one
2. Value unavailable / withheld	The value **exists** but is **deliberately withheld**	A person has a home phone but chooses not to have it listed
3. Value not applicable	The attribute simply **does not apply** to this particular tuple	LastCollegeDegree is NULL for a person who has no college degree — the attribute is undefined for them

Important practical point: It is often **impossible to determine** which of the three meanings applies just from seeing a NULL. For example, a NULL for Home_phone could mean any of the three things. **SQL does NOT distinguish** among these different meanings — it treats all NULLs uniformly at the language level.

NULLs Are Mutually Distinct

In general, each individual NULL value is considered to be different from every other NULL value in the database, even within the same column.

This has a direct consequence for comparisons: when a record with a NULL in one of its attributes is involved in a **comparison operation**, the result is **not simply TRUE or FALSE** — it is considered **UNKNOWN**.

Three-Valued Logic

Because of this, SQL uses **three-valued logic**, with possible truth values:

TRUE, FALSE, UNKNOWN

...instead of the standard **two-valued (Boolean) logic** (TRUE/FALSE only).

We therefore need to define how the logical connectives **AND**, **OR**, and **NOT** behave under three-valued logic:

Table 7.1(a) — AND

AND	**TRUE**	**FALSE**	**UNKNOWN**
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

Table 7.1(b) — OR

OR	**TRUE**	**FALSE**	**UNKNOWN**
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

Table 7.1(c) — NOT

Value	**NOT**
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Worked examples:

- FALSE AND UNKNOWN → **FALSE** (because AND with FALSE always forces FALSE, regardless of the other operand)
- FALSE OR UNKNOWN → **UNKNOWN** (because OR needs at least one TRUE to become TRUE, but FALSE doesn't resolve the UNKNOWN)

Effect on Query Results

General rule: In select-project-join queries, only combinations of tuples that make the WHERE-clause condition evaluate to **TRUE** are **selected**. Combinations evaluating to **FALSE** or **UNKNOWN** are **excluded**.

Exception: Certain operations, such as **outer joins** (Section 7.1.6), have special rules that differ from this general behavior.

Testing for NULL — IS NULL / IS NOT NULL

Since SQL treats each NULL as **distinct from every other NULL** (and from every non-NULL value), **equality comparison (= or <>) cannot be used** to test for NULL. Instead, SQL provides the special comparison operators:

IS NULL

IS NOT NULL

Important consequence: When a **join condition** is specified, tuples with **NULL values in the join attributes** are **automatically excluded** from the result — **unless** an **OUTER JOIN** is used (Section 7.1.6).

Illustrative Query

Query 18: Retrieve the names of all employees who do not have supervisors.

Q18: SELECT Fname, Lname

FROM EMPLOYEE

WHERE Super_ssn IS NULL;

5.1.2 Nested Queries, Tuples, and Set/Multiset Comparisons

Concept of Nested Queries

Some queries require **fetching existing values** from the database and then **using them in a comparison condition**. These are conveniently written using **nested queries** — a complete SELECT-FROM-WHERE block placed **inside another SQL query**.

- The **outer query** = the query that contains the nested query.
- A nested query can appear in the **WHERE clause**, the **FROM clause**, the **SELECT clause**, or other SQL clauses as needed.

The IN Comparison Operator

IN compares a value **v** with a **set (or multiset) of values V**, and evaluates to **TRUE** if v is one of the elements of V.

Illustrative Query — Using IN with Nested Queries

Query 4A: Retrieve project numbers for projects that involve an employee named 'Smith', either as manager or as worker.

```
Q4A: SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN
   ( SELECT Pnumber
     FROM PROJECT, DEPARTMENT, EMPLOYEE
     WHERE Dnum = Dnumber AND
          Mgr_ssn = Ssn AND Lname = 'Smith' )
   OR
   Pnumber IN
   ( SELECT Pno
     FROM WORKS_ON, EMPLOYEE
     WHERE Essn = Ssn AND Lname = 'Smith' );
```

Explanation:

- The **first nested query** selects project numbers of projects that have an employee named Smith as **manager**.
- The **second nested query** selects project numbers of projects that have an employee named Smith as a **worker**.
- The **outer query** uses **OR** to select any PROJECT tuple whose Pnumber matches **either** nested query's result.

Scalar vs. Set-Valued Nested Queries

- If a nested query returns a **single attribute** and a **single tuple**, the result is a **single (scalar) value** — in this case, it is permissible to use = instead of IN.
- In general, however, a nested query returns a **table (relation)** — i.e., a **set or multiset of tuples**.

Tuple Comparisons

SQL allows comparing **tuples of values** by enclosing them in **parentheses**.

Example — Find employees who work the same (project, hours) combination as employee John Smith (Ssn = '123456789'):

```
SELECT DISTINCT Essn
FROM WORKS_ON
WHERE (Pno, Hours) IN ( SELECT Pno, Hours
                        FROM WORKS_ON
                        WHERE Essn = '123456789' );
```

Here, the IN operator compares the **subtuple** (Pno, Hours) of each WORKS_ON row against the **set of type-compatible tuples** produced by the nested query.

Other Set/Multiset Comparison Operators — ANY/SOME and ALL

Beyond IN, SQL allows comparing a **single value v** to a **set/multiset V** using:

Operator	Meaning
= ANY (or **= SOME**)	TRUE if v equals **some** value in V — equivalent to IN
> ANY, **>= ANY**, **< ANY**, **<= ANY**, **<> ANY**	TRUE if the comparison holds for **at least one** value in V
> ALL, **>= ALL**, **< ALL**, **<= ALL**, **<> ALL**	TRUE if the comparison holds for **every** value in V

Note: ANY and SOME are **interchangeable keywords** with identical meaning.

Example — Employees whose salary exceeds every employee's salary in department 5:

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL ( SELECT Salary
                     FROM EMPLOYEE
                     WHERE Dno = 5 );
```

Multiple Levels of Nesting and Attribute Name Ambiguity

Queries can have **several levels of nested subqueries**. This raises the possibility of **ambiguity** when the **same attribute name** exists in both an outer-query relation and an inner (nested)-query relation.

Scope Rule: A reference to an **unqualified attribute** refers to the relation declared in the **innermost nested query** that contains that attribute. To refer to an attribute of the outer query's relation, an **alias (tuple variable)** must be used to qualify it explicitly.

Illustrative Query — Demonstrating Ambiguity

Query 16: Retrieve the name of each employee who has a dependent with the same first name and same sex as the employee.

```
Q16: SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN ( SELECT D.Essn
                 FROM DEPENDENT AS D
                 WHERE E.Fname = D.Dependent_name
                 AND E.Sex = D.Sex );
```

Why qualification matters here: In the nested query, E.Sex **must** be qualified because it refers to the Sex attribute of EMPLOYEE from the **outer** query — and DEPENDENT **also** has a Sex attribute. If Sex were left **unqualified** inside the nested query, it would default to referring to DEPENDENT.Sex (the innermost relation).

However, Fname and Ssn of EMPLOYEE **would not** need qualification even inside the nested query, because DEPENDENT has **no** attributes with those names — so there's **no ambiguity** for them.

Best practice: It is **generally advisable** to create **tuple variables (aliases)** for **all** tables referenced in a query, to avoid potential errors and ambiguity — as demonstrated in Q16.

5.1.3 Correlated Nested Queries

Definition

Whenever a condition in the **WHERE clause of a nested query** references an **attribute of a relation declared in the outer query**, the two queries are said to be **correlated**.

How to Understand a Correlated Query

A correlated nested query is conceptually evaluated **once for each tuple** (or tuple combination) of the **outer** query.

Explaining Q16 conceptually:

For **each** EMPLOYEE tuple, evaluate the nested query — retrieving the Essn values of all DEPENDENT tuples with the **same sex and name** as that EMPLOYEE tuple. If the EMPLOYEE tuple's Ssn value is found in that nested-query result, **select** that EMPLOYEE tuple.

Correlated Nested Queries Can Be Rewritten as Single-Block Queries

General rule: A query written using nested SELECT-FROM-WHERE blocks with = or IN comparison operators can **always** be re-expressed as a **single-block query** (using an explicit join).

Q16 rewritten as a single-block query (Q16A):

```
Q16A: SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn AND E.Sex = D.Sex
    AND E.Fname = D.Dependent_name;
```

Exam tip: This equivalence is important — it shows that nested and joined formulations are often **interchangeable**, giving the query writer **flexibility** in how to express the same logical query.

5.1.4 The EXISTS and UNIQUE Functions in SQL

EXISTS — Testing for a Nonempty Result

EXISTS and **UNIQUE** are **Boolean functions** returning TRUE or FALSE; hence, they can be used as conditions in a WHERE clause.

EXISTS(Q) checks whether the result of nested query **Q** is **empty or not**:

- Returns **TRUE** if the nested query result contains **at least one tuple**.
- Returns **FALSE** if the nested query result contains **no tuples**.

NOT EXISTS(Q) is the logical opposite:

- Returns **TRUE** if the nested query result is **empty**.
- Returns **FALSE** if it contains **at least one tuple**.

EXISTS/NOT EXISTS are **typically used with correlated nested queries**.

Rewriting Q16 Using EXISTS

Q16B:

```
Q16B: SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE EXISTS ( SELECT *
        FROM DEPENDENT AS D
        WHERE E.Ssn = D.Essn AND E.Sex = D.Sex
        AND E.Fname = D.Dependent_name);
```

Explanation: For each EMPLOYEE tuple, evaluate the correlated nested query. If **at least one** matching DEPENDENT tuple **EXISTS**, select that EMPLOYEE tuple.

Illustrative Query — NOT EXISTS

Query 6: Retrieve the names of employees who have no dependents.

```
Q6: SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT *
```

```
        FROM DEPENDENT
        WHERE Ssn = Essn );
```

Explanation: For each EMPLOYEE tuple, the correlated nested query selects all DEPENDENT tuples whose Essn matches the employee's Ssn. If this result is **empty** (NOT EXISTS evaluates TRUE), the employee has **no dependents**, so that EMPLOYEE tuple is selected.

Illustrative Query — Combining Multiple EXISTS Conditions

Query 7: List the names of managers who have at least one dependent.

```
Q7: SELECT Fname, Lname
FROM EMPLOYEE
WHERE EXISTS ( SELECT *
        FROM DEPENDENT
        WHERE Ssn = Essn )
    AND
    EXISTS ( SELECT *
        FROM DEPARTMENT
        WHERE Ssn = Mgr_ssn );
```

Explanation: Two correlated nested queries are used:

- The **first** checks whether the employee **has at least one dependent**.
- The **second** checks whether the employee **manages some department**.

Both must be **TRUE** (via AND) for the EMPLOYEE tuple to be selected.

Universal Quantification Queries — Using EXCEPT / EXISTS-NOT EXISTS

Some queries require **universal quantification** ("for ALL X, condition holds") — these are among the **trickiest** SQL queries to formulate.

Query 3: Retrieve the name of each employee who works on ALL the projects controlled by department number 5.

Method 1 — Using EXCEPT (set difference):

```
Q3A: SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS ( ( SELECT Pnumber
                     FROM PROJECT
                     WHERE Dnum = 5)
                   EXCEPT ( SELECT Pno
                            FROM WORKS_ON
                            WHERE Ssn = Essn) );
```

Explanation:

- **EXCEPT** is the **set difference** operator (in Oracle, the equivalent keyword is **MINUS**).
- The **first subquery** (not correlated) selects **all** projects controlled by department 5.
- The **second subquery** (correlated) selects all projects that the **specific employee** currently being considered works on.
- **If (Set of Dept-5 projects) MINUS (Set of employee's projects) = empty**, it means the employee works on **every** project controlled by department 5 — so the employee is selected via NOT EXISTS.

Method 2 — Using nested EXISTS/NOT EXISTS (more complex, two-level nesting):

```
Q3B: SELECT Lname, Fname
FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT *
                   FROM WORKS_ON B
                   WHERE ( B.Pno IN ( SELECT Pnumber
                                      FROM PROJECT
                                      WHERE Dnum = 5 )
```

```
AND
NOT EXISTS ( SELECT *
        FROM WORKS_ON C
        WHERE C.Essn = Ssn
        AND C.Pno = B.Pno )));
```

Explanation (rephrasing of Query 3): *"Select each employee such that there does NOT exist a project controlled by department 5 that the employee does NOT work on."*

- The outer nested query selects any WORKS_ON (B) tuple whose Pno belongs to a department-5 project, **provided there is no** matching WORKS_ON (C) tuple with the **same** Pno **and** the **same** Ssn as the employee under consideration.
- If **no such B tuple exists** at all, the employee is selected.
- This corresponds directly to how this type of query is expressed in **tuple relational calculus** using universal quantifiers (Section 8.6.7).

Exam Tip: "Works on ALL projects of department X" is a classic **universal quantification** pattern. Remember the **double-negation trick**: *NOT EXISTS (a project the employee does NOT work on)* is the standard SQL idiom for expressing "works on ALL projects."

The UNIQUE Function

UNIQUE(Q) returns **TRUE** if there are **no duplicate tuples** in the result of query Q; otherwise, it returns **FALSE**.

This is used to test whether the result of a nested query is a **set** (no duplicates) versus a **multiset** (duplicates present).

5.1.5 Explicit Sets and Renaming in SQL

Explicit Sets

Instead of a nested query, SQL allows specifying an **explicit set of values** directly in the WHERE clause, enclosed in **parentheses**.

Query 17: Retrieve the Ssns of all employees who work on project numbers 1, 2, or 3.

Q17: SELECT DISTINCT Essn

FROM WORKS_ON

WHERE Pno IN (1, 2, 3);

Renaming (Aliasing) with AS

SQL allows **renaming** any attribute appearing in the result of a query, using the AS qualifier followed by the desired new name. **AS** can be used to alias **both attribute names and relation names**, and the new names appear as **column headers** in the query result.

Example (Q8A):

Q8A: SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name

FROM EMPLOYEE AS E, EMPLOYEE AS S

WHERE E.Super_ssn = S.Ssn;

This retrieves each employee's last name and their supervisor's last name, renaming the output columns as Employee_name and Supervisor_name.

5.1.6 Joined Tables in SQL and Outer Joins

The Concept of a Joined Table

SQL incorporates the concept of a **joined table (joined relation)**, allowing a table resulting from a **JOIN operation** to be specified **directly in the FROM clause**. This can be **easier to understand** than mixing all select and join conditions together in the WHERE clause.

Example — Q1A (using JOIN...ON in the FROM clause):

Q1A: SELECT Fname, Lname, Address

FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)

WHERE Dname = 'Research';

Here, the **FROM clause** contains a **single joined table**, whose attributes = all attributes of EMPLOYEE **followed by** all attributes of DEPARTMENT.

NATURAL JOIN

In a **NATURAL JOIN** of relations R and S:

- **No explicit join condition is specified.**
- SQL implicitly creates an **EQUIJOIN condition** for **every pair of attributes with the same name** in R and S.
- Each such matching attribute pair appears **only once** in the resulting relation (no duplicate columns).

Renaming for NATURAL JOIN Compatibility

If the join attributes **do not share the same name** in the base relations, they can be **renamed** so a NATURAL JOIN can be applied. The AS construct can rename **a relation and all its attributes** simultaneously in the FROM clause.

Example — Q1B:

Q1B: SELECT Fname, Lname, Address

FROM (EMPLOYEE NATURAL JOIN

(DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))

WHERE Dname = 'Research';

Here, DEPARTMENT is renamed as DEPT, and its attributes renamed to (Dname, Dno, Mssn, Msdate) — specifically renaming Dnumber to Dno so it **matches** the EMPLOYEE.Dno attribute name. The **implied join condition** becomes EMPLOYEE.Dno = DEPT.Dno, since that's the only matching attribute-name pair after renaming.

Inner Join (Default) vs. Outer Join

The **default type of join** in a joined table is called an **INNER JOIN** — a tuple is included in the result **only if a matching tuple exists** in the other relation.

Example of inner-join limitation: In Q8A (the employee/supervisor self-join), only employees **who have a supervisor** are included — an employee with Super_ssn = NULL is **excluded**.

To include ALL employees (even those without a supervisor), an **OUTER JOIN** must be used explicitly.

Types of Outer Joins

Example — Q8B using LEFT OUTER JOIN:

Q8B: SELECT E.Lname AS Employee_name,

S.Lname AS Supervisor_name

FROM (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S

ON E.Super_ssn = S.Ssn);

Join Type	Behavior
INNER JOIN (same as plain JOIN)	Only pairs of tuples that **satisfy** the join condition are retrieved
LEFT OUTER JOIN	**Every tuple in the LEFT table** must appear in the result. If it has **no matching** tuple in the right table, it is **padded with NULLs** for the right table's attributes
RIGHT OUTER JOIN	**Every tuple in the RIGHT table** must appear in the result; unmatched left-table attributes are **padded with NULLs**
FULL OUTER JOIN	**Every tuple from BOTH tables** must appear; unmatched attributes on either side are padded with **NULLs**

Note: For LEFT OUTER JOIN, RIGHT OUTER JOIN, and FULL OUTER JOIN, the keyword **OUTER may be omitted** (e.g., simply LEFT JOIN is valid).

NATURAL Variation of Outer Joins

If the join attributes share the **same name**, the **NATURAL** keyword can be combined with outer join types:

NATURAL LEFT OUTER JOIN

CROSS JOIN

CROSS JOIN specifies the **CARTESIAN PRODUCT** operation.

Caution: This should be used with **utmost care**, since it generates **all possible combinations** of tuples from both relations — resulting in a potentially enormous result set.

Multiway Joins (Nested Join Specifications)

It is possible to **nest** join specifications — i.e., one of the "tables" in a join can itself **be a joined table**. This allows specifying the join of **three or more tables** as a single joined table, called a **multiway join**.

Example — Q2A:

Q2A: SELECT Pnumber, Dnum, Lname, Address, Bdate

FROM ((PROJECT JOIN DEPARTMENT ON Dnum = Dnumber)

JOIN EMPLOYEE ON Mgr_ssn = Ssn)

WHERE Plocation = 'Stafford';

Here, PROJECT is first joined with DEPARTMENT, and the **result** of that join is then joined with EMPLOYEE — forming a **3-way join** in a single nested FROM expression.

Alternative (Legacy) Outer Join Syntax

Not all SQL implementations support the newer standard joined-table syntax. Some systems (e.g., **Oracle**) historically used special comparison operators for outer joins:

Operator	Meaning
+ =	LEFT outer join
= +	RIGHT outer join
+ = +	FULL outer join

Example — Q8C (Oracle-style legacy syntax):

Q8C: SELECT E.Lname, S.Lname

FROM EMPLOYEE E, EMPLOYEE S

WHERE E.Super_ssn + = S.Ssn;

5.1.7 Aggregate Functions in SQL

Purpose

Aggregate functions summarize information from **multiple tuples** into a **single summary value**. **Grouping** is used to create **subgroups** of tuples **before**

summarization — grouping and aggregation together support many real database applications.

Built-in Aggregate Functions

Function	Description
COUNT	Returns the number of tuples or values specified
SUM	Returns the sum of a set/multiset of numeric values
MAX	Returns the maximum value
MIN	Returns the minimum value
AVG	Returns the average (mean) of numeric values

Notes:

- These functions can appear in the SELECT clause **or** in a HAVING clause.
- MAX and MIN can also be applied to **nonnumeric domains**, provided the domain has a **total ordering** (e.g., DATE, TIME, TIMESTAMP, or alphabetic strings all have total orderings).
- *(SQL-99 added further statistical aggregate functions for advanced calculations.)*

Illustrative Queries

Query 19: Find the sum, maximum, minimum, and average of all employee salaries.

Q19: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)

FROM EMPLOYEE;

This returns a **single-row summary** across the entire table.

With renamed columns (Q19A):

Q19A: SELECT SUM (Salary) AS Total_Sal, MAX (Salary) AS Highest_Sal,

MIN (Salary) AS Lowest_Sal, AVG (Salary) AS Average_Sal

FROM EMPLOYEE;

Query 20: Find the same aggregate values, but restricted to the 'Research' department.

Q20: SELECT SUM (Salary), MAX (Salary), MIN (Salary), AVG (Salary)

FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)

WHERE Dname = 'Research';

Queries 21 & 22: Count total employees, and employees in 'Research'.

Q21: SELECT COUNT (*)

FROM EMPLOYEE;

Q22: SELECT COUNT (*)

FROM EMPLOYEE, DEPARTMENT

WHERE DNO = DNUMBER AND DNAME = 'Research';

Note on COUNT(*): The asterisk * refers to **rows (tuples)** — so COUNT(*) counts the number of **rows** in the query result, **including** duplicates and rows with NULLs in any column.

Query 23: Count the number of distinct salary values.

Q23: SELECT COUNT (DISTINCT Salary)

FROM EMPLOYEE;

Behavior of Aggregate Functions With NULLs — Important Rule

General rule: When an aggregate function is applied to a collection of values, **NULLs are first removed** from the collection **before** the calculation.

- If the collection becomes **empty** because **all** values were NULL, the aggregate function returns **NULL** — **EXCEPT** for COUNT, which returns **0** for an empty collection.

Specific notes:

- COUNT(Salary) (without DISTINCT) does **not** eliminate duplicate values, but **does** exclude NULL salary values from the count.
- COUNT(*) is the **only exception** to the "NULLs are removed" rule, because it counts **tuples**, not individual attribute values.

Aggregate Functions in Correlated Nested Queries

Aggregate functions can be used **inside** correlated nested queries used within an outer WHERE clause.

Query 5: Retrieve names of employees who have two or more dependents.

```
Q5: SELECT Lname, Fname
FROM EMPLOYEE
WHERE ( SELECT COUNT (*)
        FROM DEPENDENT
        WHERE Ssn = Essn ) >= 2;
```

Explanation: The correlated nested query counts each employee's dependents; if the count is **≥ 2**, that EMPLOYEE tuple is selected.

Boolean Aggregate Functions — SOME and ALL

SQL also provides SOME and ALL as **aggregate functions applied to Boolean values**:

- **SOME** → returns TRUE if **at least one** element in the collection is TRUE.
- **ALL** → returns TRUE if **every** element in the collection is TRUE.

5.1.8 Grouping: The GROUP BY and HAVING Clauses

Why Grouping Is Needed

Often we want to apply aggregate functions to **subgroups** of tuples based on some attribute value — e.g., "average salary **per department**" or "number of employees **per project**." This requires **partitioning** the relation into **nonoverlapping groups**, where each group shares the same value(s) of some **grouping attribute(s)**.

SQL's **GROUP BY** clause specifies the grouping attribute(s). These attributes should also appear in the SELECT clause, so the result of each aggregate function (per group) is shown **alongside** the corresponding grouping attribute value(s).

Illustrative Query — Basic GROUP BY

Query 24: For each department, retrieve the department number, employee count, and average salary.

Q24: SELECT Dno, COUNT (*), AVG (Salary)

FROM EMPLOYEE

GROUP BY Dno;

Explanation: EMPLOYEE tuples are **partitioned into groups**, each group sharing the same Dno value. COUNT and AVG are then applied **independently to each group**.

NULLs in the Grouping Attribute

If NULLs exist in the grouping attribute, a **separate group** is created for **all tuples with a NULL** value in that attribute.

GROUP BY With a Join Condition

Query 25: For each project, retrieve project number, name, and count of employees working on it.

Q25: SELECT Pnumber, Pname, COUNT (*)

FROM PROJECT, WORKS_ON

WHERE Pnumber = Pno

GROUP BY Pnumber, Pname;

Explanation: Grouping and aggregation are applied **after** the join between PROJECT and WORKS_ON is performed in the WHERE clause.

The HAVING Clause — Filtering Groups

Sometimes we want summary results **only for groups that satisfy a certain condition**. The **HAVING** clause provides a condition applied to **summary/group information**, and appears **in conjunction with GROUP BY**.

Only the **groups** satisfying the HAVING condition are included in the final result.

Query 26: For each project with MORE THAN TWO employees, retrieve project number, name, and employee count.

Q26: SELECT Pnumber, Pname, COUNT (*)

FROM PROJECT, WORKS_ON

WHERE Pnumber = Pno

```
GROUP BY Pnumber, Pname
HAVING COUNT (*) > 2;
```

Critical distinction:

While WHERE-clause conditions **restrict individual tuples** before grouping, the HAVING clause serves to **choose entire groups** after grouping.

GROUP BY With Both WHERE and Aggregate

Query 27: For each project, count only the employees FROM department 5 working on it.

```
Q27: SELECT Pnumber, Pname, COUNT (*)
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Ssn = Essn AND Dno = 5
GROUP BY Pnumber, Pname;
```

Here, the WHERE clause restricts tuples **before** grouping — so each group's COUNT only reflects **department-5 employees**.

Critical Caution — Order of WHERE vs. HAVING Evaluation

The WHERE clause is executed FIRST, to select individual (or joined) tuples. **The HAVING clause is applied AFTER**, to select whole groups based on already-computed summary values.

Common mistake — an INCORRECT query:

Suppose we want: *"count employees earning > $40,000 per department, but only for departments having more than 5 employees total."*

```
-- INCORRECT
SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000
GROUP BY Dno
HAVING COUNT (*) > 5;
```

Why this is wrong: Because WHERE Salary > 40000 **filters tuples BEFORE grouping**, by the time HAVING COUNT(*) > 5 runs, it's counting **only high earners** — so this actually selects departments having **more than 5 employees who EACH earn over $40,000**, which is a **completely different (and unintended) condition**.

Correct approach — Query 28, using a nested subquery:

Query 28: For each department with more than 5 employees, retrieve the department number and the count of its employees earning more than $40,000.

```
Q28: SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN
   ( SELECT Dno
     FROM EMPLOYEE
     GROUP BY Dno
     HAVING COUNT (*) > 5)
GROUP BY Dno;
```

Explanation: The **inner nested query** first identifies departments having **more than 5 employees** (using their **own** independent GROUP BY ... HAVING). The **outer query** then filters EMPLOYEE tuples to those in one of these "big" departments **AND** earning over $40,000, before grouping and counting.

Exam Tip: Whenever a query needs **two different conditions** — one that applies to the **aggregate/count in SELECT** and another that applies to a **different aggregate condition** (like "department size") — you generally need a **nested subquery**, because a single WHERE+HAVING pair cannot independently apply two different filtering criteria at two different aggregation stages.

5.1.9 Other SQL Constructs: WITH and CASE

The WITH Clause

The **WITH** clause allows defining a **temporary named table**, used **only within a particular query** — conceptually similar to creating a **view** (Section 7.3) that's used once and then discarded.

- Introduced in **SQL:99** as a **convenience** feature.
- **May not be available** in all SQL-based DBMSs.
- Any query using WITH can generally be **rewritten** using other standard SQL constructs (like nested subqueries).

Example — Rewriting Q28 using WITH (Q28′):

```
Q28': WITH BIGDEPTS (Dno) AS
   ( SELECT Dno
     FROM EMPLOYEE
     GROUP BY Dno
     HAVING COUNT (*) > 5)
SELECT Dno, COUNT (*)
FROM EMPLOYEE
WHERE Salary>40000 AND Dno IN BIGDEPTS
GROUP BY Dno;
```

Explanation: The WITH clause defines a **temporary table** BIGDEPTS, holding the department numbers with more than 5 employees. This temporary table is then used in the **main query**. Once the query finishes executing, BIGDEPTS is **discarded**.

The CASE Construct

The **CASE** construct allows a value to **differ based on conditions**. It can be used **anywhere** a value is expected in an SQL statement — including in SELECT queries, INSERT, or UPDATE operations.

Example — Different salary raises based on department:

Suppose: department 5 employees get a **$2,000** raise, department 4 get **$1,500**, and department 1 get **$3,000**.

```
U6': UPDATE EMPLOYEE
SET Salary =
CASE WHEN Dno = 5 THEN Salary + 2000
```

```
    WHEN Dno = 4 THEN Salary + 1500
    WHEN Dno = 1 THEN Salary + 3000
    ELSE Salary + 0
END;
```

Explanation: The raise amount is **conditionally determined** based on Dno via the CASE construct, all within a **single UPDATE statement**.

Other use case: CASE is also useful when **inserting** tuples where different attributes should be NULL depending on the **type of record** — e.g., when a specialization/subclass hierarchy (Chapter 4) is mapped into a **single table** (Chapter 9), or when a **union type** is mapped into relations.

5.1.10 Recursive Queries in SQL

The Need for Recursion

A **recursive relationship** relates tuples of a relation **to other tuples of the same relation**. A classic example: the **employee–supervisor** relationship, represented by the **foreign key** Super_ssn in the EMPLOYEE relation — this relates each employee (as **supervisee**) to another employee (as **supervisor**).

Example of a recursive operation: Retrieve **all** supervisees of a supervisory employee **e**, at **all levels** — i.e.:

- All employees **directly** supervised by e (level 1),
- All employees directly supervised by **those** employees (level 2),
- And so on, **recursively**, through all levels.

SQL:99 Recursive Query Syntax

This capability was added in **SQL:99** using the WITH RECURSIVE construct, allowing recursive queries to be expressed **declaratively**.

Query 29 — Retrieve all supervisees at all levels:

```
Q29: WITH RECURSIVE SUP_EMP (SupSsn, EmpSsn) AS
   ( SELECT SupervisorSsn, Ssn
     FROM EMPLOYEE
```

```
    UNION
    SELECT E.Ssn, S.SupSsn
    FROM EMPLOYEE AS E, SUP_EMP AS S
    WHERE E.SupervisorSsn = S.EmpSsn)
SELECT *
FROM SUP_EMP;
```

How the Recursion Works

1. SUP_EMP is defined as a **view/temporary table** that will hold the recursive query's result. It starts out **empty**.
2. The **base query** — SELECT SupervisorSsn, Ssn FROM EMPLOYEE — loads the **first level** of (supervisor, supervisee) pairs.
3. This is combined via **UNION** with the **second part**, which **joins** the current SUP_EMP contents with EMPLOYEE again, to compute the **next level** of supervisee relationships.
4. This process **repeats iteratively** — each iteration adding the **next level** of supervisees — **until a fixed point is reached** (i.e., no new tuples are added).
5. At that point, SUP_EMP contains the **complete transitive closure** of the supervisor-supervisee relationship — supervisees at **every level**.

5.1.11 Discussion and Summary of SQL Queries

The Complete SQL Query Structure

A retrieval query in SQL can have **up to six clauses**, but **only the first two are mandatory**:

```
SELECT <attribute and function list>
FROM <table list
[ WHERE <condition> ]
[ GROUP BY <grouping attribute(s)> ]
[ HAVING <group condition> ]
```

[ORDER BY <attribute list>];

Notes on syntax:

- The query can **span multiple lines** and is terminated by a **semicolon**.
- Query terms are separated by **spaces**; **parentheses** group relevant parts as needed.
- Clauses in **square brackets [ ]** are **optional**.

Purpose of Each Clause

Clause	Purpose
SELECT	Lists the attributes/functions to be **retrieved**
FROM	Specifies **all relations** needed (including joined relations), **excluding** those used only in nested queries
WHERE	Specifies **conditions** for selecting tuples, including **join conditions**
GROUP BY	Specifies the **grouping attribute(s)**
HAVING	Specifies a condition on the **groups** (summary info), not on individual tuples
ORDER BY	Specifies the **display order** of the final result

Note on aggregate functions: COUNT, SUM, MIN, MAX, AVG are typically used with GROUP BY, but they can **also** be applied to **all** selected tuples **without** a GROUP BY clause (treating the entire result as one group).

Conceptual Order of Query Evaluation

To formulate queries correctly, it helps to think of the **conceptual (logical) evaluation order** of the clauses — **note this is NOT necessarily the actual physical execution order**, which is **implementation-dependent** and optimized by the DBMS:

1. FROM — identify all tables/materialize joined tables
2. WHERE — select and join tuples

3. GROUP BY — partition into groups

4. HAVING — filter groups

5. SELECT — compute output attributes/aggregates

6. ORDER BY — sort the final result

Simplified conceptual execution (if no GROUP BY/HAVING/ORDER BY):

For **each combination** of tuples — one from each relation named in the FROM clause evaluate the WHERE-clause condition. If it evaluates to **TRUE**, place the SELECT-clause attribute values from this combination into the result.

Multiple Ways to Express the Same Query — Trade-offs

SQL offers great flexibility: the **same** logical query can typically be written in **numerous different ways** — using joins in the WHERE clause, using **joined relations** in the FROM clause, or using **nested queries** with IN, EXISTS, etc.

Advantages of this flexibility:

- Users can choose the technique they are **most comfortable** with.
- Different users may prefer different styles for expressing the same logic.

Disadvantages:

- This flexibility can **confuse** users who don't know which technique to use for a particular query type.
- **Performance may differ** between equivalent formulations — even though, *ideally*, a DBMS should process logically equivalent queries **identically regardless of syntax**, in practice this is **difficult to guarantee**, since different DBMSs use different internal query-processing methods for different query forms.

Practical guidance: From a programmer's and query-optimizer's perspective, it is **generally preferable** to write queries with **as little nesting and implied ordering as possible** — simpler, flatter queries tend to be **easier for the optimizer to process efficiently**.

Ideal vs. Practical Responsibility:

Ideally, the **user** should worry only about specifying the query **correctly**, while the **DBMS** determines how to execute it **efficiently**. In practice, however, it helps for users to be **aware** of which SQL constructs tend to be **more expensive** to process than others.

1. **NULL comparisons never yield simple TRUE/FALSE** — they yield UNKNOWN; only TRUE rows are selected.
2. **Use IS NULL/IS NOT NULL**, never = NULL.
3. **IN** = equivalent to = ANY/= SOME.
4. **Correlated nested query** = references an outer-query attribute; conceptually re-evaluated per outer tuple.
5. **EXISTS** = TRUE if nested query returns ≥1 row; **NOT EXISTS** = TRUE if it returns 0 rows.
6. **"Works on ALL X"** type queries need **universal quantification** — typically solved via EXCEPT (set difference = empty) or double NOT EXISTS.
7. **WHERE filters tuples BEFORE grouping; HAVING filters GROUPS after aggregation** — never confuse the two.
8. **Aggregate functions ignore NULLs** (except COUNT(*), which counts rows including those with NULLs, and returns 0 — not NULL — for an empty group).
9. **LEFT/RIGHT/FULL OUTER JOIN** preserves unmatched rows from one/both sides, padding with NULLs.
10. **WITH RECURSIVE** is needed for hierarchical queries with unknown/variable depth.

5.2 Specifying Constraints as Assertions and Actions as Triggers

Introduction

This section introduces **two additional, powerful features** of SQL beyond ordinary DDL/DML:

1. **CREATE ASSERTION** — for specifying **general constraints** that go beyond the built-in relational model constraints.

2. **CREATE TRIGGER** — for specifying **automatic actions** that the database system performs in response to certain events and conditions. This functionality is generally referred to as **active databases**.

5.2.1 Specifying General Constraints as Assertions in SQL

Purpose

In SQL, users can specify **general constraints** — those that do **not** fall into any of the standard categories covered in Sections 6.1 and 6.2 (i.e., not simple key, NOT NULL, domain, or referential integrity constraints) — using **declarative assertions**, via the **CREATE ASSERTION** statement.

- Each assertion is given a **constraint name**.
- It is specified via a **condition**, syntactically similar to a WHERE clause of an SQL query.

Illustrative Example

Constraint: *"The salary of an employee must not be greater than the salary of the manager of the department that employee works for."*

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK ( NOT EXISTS ( SELECT *
                     FROM EMPLOYEE E, EMPLOYEE M,
                         DEPARTMENT D
                     WHERE E.Salary > M.Salary
                     AND E.Dno = D.Dnumber
                     AND D.Mgr_ssn = M.Ssn ) );
```

Syntax Breakdown

- **SALARY_CONSTRAINT** — the **constraint name**, given right after CREATE ASSERTION. This name can later be used to **disable**, **modify**, or **drop** the constraint.
- **CHECK** — followed by a **condition in parentheses** that must **hold true on every database state** for the assertion to be satisfied.

- The **DBMS is responsible** for continuously ensuring this condition is never violated, across all database operations.

The General Technique for Writing Assertions

The basic technique is to write a **query that selects any tuples violating the desired condition**, and wrap it inside a **NOT EXISTS** clause.

- The assertion specifies that the result of this "violation-detecting" query **must be empty** for the condition to remain TRUE.
- **Thus:** the assertion is **violated** if and only if the result of the inner query is **NOT empty**.

Applying this to our example: The inner query selects **all employees whose salary exceeds their department manager's salary**. If this query's result is **non-empty**, the assertion is **violated**.

General Rule for Evaluating Assertions

Whenever some tuples in the database cause the condition of an ASSERTION statement to evaluate to **FALSE**, the constraint is **violated**. The constraint is **satisfied** by a database state if **no combination of tuples** in that state violates the constraint.

Any valid WHERE-clause condition can be used inside CHECK, but **many** general constraints are conveniently expressed using the **EXISTS / NOT EXISTS** style shown above.

CREATE ASSERTION vs. Attribute/Domain/Tuple CHECK Clauses

The CHECK clause can also be used to specify constraints on:

- **Individual attributes and domains**
- **Individual tuples**

Key difference — WHEN each type of CHECK is evaluated:

Constraint Type	Checked When
CHECK on individual attributes/domains/tuples	Only when tuples are **inserted or updated** in **that specific table**

Constraint Type	Checked When
CREATE ASSERTION	Must hold on **every** database state — potentially involves **multiple tables**, and must be checked whenever **any relevant table** changes

Consequence: Since attribute/tuple-level CHECK clauses are scoped to a **single table** and evaluated **only** on insert/update of that table, the DBMS can implement them **more efficiently**.

Design Guideline — When to Use Which

- The schema designer should use **CHECK on attributes, domains, and tuples only** when they are certain the constraint can be violated **only** by an insertion or update to that **single** table.
- The schema designer should use **CREATE ASSERTION only** in cases where it is **not possible** to express the constraint using attribute/domain/tuple-level CHECK — since simpler checks are implemented **more efficiently** by the DBMS.

Exam Tip: CREATE ASSERTION is more **powerful and general** (it can span multiple tables and be evaluated on any relevant change), but it is also typically **more expensive** for the DBMS to enforce than a localized CHECK constraint. Use the "cheapest" mechanism that can express the constraint.

7.2.2 Introduction to Triggers in SQL

Motivation

It is often useful to specify that the DBMS should **automatically take some action** when certain **events** occur and certain **conditions** are satisfied.

Example scenario: A manager wants to be **automatically informed** whenever an **employee's travel expenses exceed a certain limit**. Here:

- The **condition** ("expenses exceed limit") is used to **monitor** the database.
- The **action** the DBMS must take is to **send a message** to the manager.

Other possible actions include: **executing a stored procedure**, or **triggering additional updates** elsewhere in the database.

The **CREATE TRIGGER** statement is used to implement such automatic event-driven actions in SQL. This functionality is the foundation of what's called **active databases** (discussed fully in Section 26.1).

Illustrative Example — SALARY_VIOLATION Trigger

Scenario: In the COMPANY database, we want to automatically check whenever an **employee's salary becomes greater than their direct supervisor's salary**.

Possible triggering events:

1. **Inserting** a new employee record
2. **Changing** an employee's salary
3. **Changing** an employee's supervisor

Desired action: Call an external stored procedure SALARY_VIOLATION (or INFORM_SUPERVISOR) to **notify the supervisor**.

Trigger definition (using Oracle syntax):

```
R5: CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF SALARY, SUPERVISOR_SSN
   ON EMPLOYEE
FOR EACH ROW
   WHEN ( NEW.SALARY > ( SELECT SALARY FROM EMPLOYEE
                  WHERE SSN = NEW.SUPERVISOR_SSN ) )
   INFORM_SUPERVISOR(NEW.Supervisor_ssn,
                NEW.Ssn );
```

- **SALARY_VIOLATION** — the **trigger name**, which can be used later to **remove or deactivate** the trigger.

The ECA (Event-Condition-Action) Rule Model

A typical trigger is regarded as an **ECA (Event, Condition, Action) rule**, consisting of **three components**:

1. The Event(s)

- These are usually **database update operations** explicitly applied to the database.
- **In this example, the events are:** inserting a new employee record, changing an employee's salary, **or** changing an employee's supervisor.
- **Important responsibility:** The trigger writer **must ensure all possible relevant events are accounted for** — in some cases, this might require writing **more than one trigger** to cover all scenarios completely.
- **Timing keyword:**
  - **BEFORE** — specifies the trigger should execute **before** the triggering operation is actually carried out (used in our example).
  - **AFTER** — an alternative keyword specifying the trigger should execute **after** the triggering event has completed.

2. The Condition

- Once the triggering **event** has occurred, an **optional condition** may be evaluated.
- **If no condition is specified:** the action executes **immediately** once the event occurs.
- **If a condition IS specified:** it is evaluated **first**, and the rule's **action** executes **only if** the condition evaluates to **TRUE**.
- **Syntax:** The condition is specified in the **WHEN** clause of the trigger.
- In our example: WHEN (NEW.SALARY > (SELECT SALARY FROM EMPLOYEE WHERE SSN = NEW.SUPERVISOR_SSN)) — checks whether the **new (about-to-be-inserted/updated) salary** exceeds the **supervisor's current salary**.

3. The Action

- The action is usually a **sequence of SQL statements**, but it could also be a full **database transaction** or an **external program** that gets **automatically executed**.

- In our example: the action is to **execute the stored procedure** INFORM_SUPERVISOR, passing the supervisor's Ssn and the employee's Ssn as parameters.

Special Keywords — NEW and (implicitly) OLD

- **NEW.SALARY**, **NEW.SUPERVISOR_SSN**, **NEW.Ssn** — these refer to the **new/updated values** of the row being inserted or updated, i.e., the value **after** the triggering operation would take effect.
- **FOR EACH ROW** — specifies that this is a **row-level trigger**: the trigger logic executes **once for every row** affected by the triggering INSERT/UPDATE operation (as opposed to a **statement-level** trigger, which executes just once per SQL statement, regardless of how many rows it affects).

Applications of Triggers

Triggers can be used in various applications, such as:

- **Maintaining database consistency**
- **Monitoring database updates**
- **Automatically updating derived data**
-

5.3 Views (Virtual Tables) in SQL

1. Introduction to Views

Definition

A **View** in SQL is a **virtual table** that is created using one or more existing tables.

Unlike normal tables, a view **does not physically store data**.

Instead, it stores **only the SQL query** used to generate the data.

Whenever the view is accessed, SQL executes the stored query and displays the latest data from the original tables.

Hence,

View = Stored SQL Query + Virtual Table

Definition

A View is a virtual table whose contents are derived from one or more base tables using an SQL query. It does not store data permanently but displays data obtained from the underlying tables whenever it is queried.

Base Table vs View

Base Table	View
Physically stored	Virtual table
Stores actual data	Stores only SQL query
Occupies storage	Very little storage
Can always be updated	Update restrictions exist
Created using CREATE TABLE	Created using CREATE VIEW

Why do we need Views?

Suppose a company database contains three tables:

- EMPLOYEE
- PROJECT
- WORKS_ON

If every employee report requires joining all three tables,

every query becomes long.

Example:

```
SELECT Fname,Lname,Pname,Hours
FROM EMPLOYEE,PROJECT,WORKS_ON
WHERE
Ssn=Essn
AND Pno=Pnumber;
```

Instead of writing this repeatedly,

we create one view.

Then simply write

SELECT *

FROM WORKS_ON1;

This makes SQL easier.

Advantages of Views

1. Simplifies Complex Queries

Complex joins are hidden.

Instead of writing long joins,

users simply query the view.

Example

Instead of

SELECT...

FROM A,B,C

WHERE...

User writes

SELECT *

FROM VIEW_NAME;

2. Provides Security

Sensitive columns can be hidden.

Example

Instead of showing

- Salary
- Age
- Department

A view may show only

- Name
- Address

Unauthorized users cannot access hidden columns.

3. Restricts Rows

Views can display only required records.

Example

Instead of showing all employees,

View shows only

Department 5 employees.

4. Data Independence

If underlying tables change,

view can often remain unchanged.

Applications continue working.

5. Easy Data Retrieval

Frequently used reports become easier.

No need to remember long SQL joins.

Characteristics of Views

- Virtual table
- Derived from one or more tables
- Does not permanently store tuples
- Automatically reflects latest database changes
- Can be queried like normal tables
- May or may not be updatable

CREATE VIEW Syntax

General Syntax

CREATE VIEW view_name

AS

SELECT column1,column2,...

FROM table_name

WHERE condition;

Example 1

CREATE VIEW WORKS_ON1

AS

SELECT

Fname,

Lname,

Pname,

Hours

FROM

EMPLOYEE,

PROJECT,

WORKS_ON

WHERE

Ssn=Essn

AND

Pno=Pnumber;

Explanation

This view joins

- EMPLOYEE

- PROJECT
- WORKS_ON

and displays

- Employee First Name
- Employee Last Name
- Project Name
- Hours Worked

Instead of repeatedly joining three tables,

we simply query

```
SELECT *
FROM WORKS_ON1;
```

Example 2

```
CREATE VIEW DEPT_INFO
(Dept_name,
No_of_emps,
Total_sal)
AS
SELECT
Dname,
COUNT(*),
SUM(Salary)
FROM
DEPARTMENT,
EMPLOYEE
WHERE
```

Dnumber=Dno

GROUP BY

Dname;

Explanation

This view displays

Department Name

↓

Number of Employees

↓

Total Salary

It uses

- COUNT()
- SUM()
- GROUP BY

Therefore,

this view is based on aggregate functions.

Querying a View

Views behave exactly like tables.

Example

SELECT

Fname,

Lname

FROM

WORKS_ON1

WHERE

Pname='ProductX';

Output

Employees working on ProductX.

Notice

User never joins three tables.

The view performs that automaticall.

DROP VIEW

If the view is no longer required

DROP VIEW WORKS_ON1;

This removes only the view.

Original tables remain unchanged.

View Implementation

A DBMS can implement views in two different ways.

Method 1 : Query Modification

Also called

View Expansion

Working

The DBMS replaces the view with its original SQL query.

Example

User writes

SELECT *

FROM WORKS_ON1;

DBMS converts it into

SELECT

Fname,

Lname,

Pname,

Hours

FROM

EMPLOYEE,

PROJECT,

WORKS_ON

WHERE

Ssn=Essn

AND

Pno=Pnumber;

Then executes it.

Advantages

- No extra storage
- Always latest data
- Simple implementation

Disadvantages

Every query performs joins again.

Slow for large databases.

Method 2 : View Materialization

Instead of executing joins every time,

DBMS stores the result temporarily.

This stored result is called

Materialized View.

Future queries read directly from this stored table.

Advantages

Very fast.

Good for repeated queries.

Disadvantages

Whenever original tables change,

materialized view must also be updated.

Updating Materialized Views

Three strategies exist.

1. Immediate Update

View updates immediately after base table changes.

Advantages

Always current.

Disadvantage

High overhead.

2. Lazy Update

View updated only when someone accesses it.

Advantages

Less overhead.

Disadvantage

May temporarily contain outdated data.

3. Periodic Update

View updates after fixed intervals.

Example

Every night

Every hour

Every week

Advantages

Efficient.

Disadvantage

Users may see old data until refresh occurs.

Updating Views

Users can perform

- INSERT
- UPDATE
- DELETE

only under certain conditions.

Not every view is updatable.

Example of Ambiguous Update

Suppose

UPDATE WORKS_ON1

SET

Pname='ProductY'

WHERE

Fname='John'

AND

Lname='Smith'

AND

Pname='ProductX';

Question

Should SQL

- Change project assignment?

OR

- Rename ProductX into ProductY?

Both produce same visible result.

Hence DBMS cannot decide.

Therefore,

updates on join views are generally not allowed.

Non-Updatable Views

Views cannot usually be updated when they contain

- Multiple table joins
- Aggregate functions
- GROUP BY
- SUM()
- COUNT()
- AVG()
- MIN()
- MAX()

Example

UPDATE DEPT_INFO

SET

Total_sal=100000;

Impossible.

Because

Total_sal is calculated using

SUM(Salary)

It is not directly stored.

Hence update is rejected.

Updatable Views

Generally,

a view is updatable when

- Only one table is used.
- Primary key is present.
- NOT NULL attributes are included.
- No aggregate functions.
- No GROUP BY.

WITH CHECK OPTION

Syntax

CREATE VIEW view_name

AS

SELECT ...

FROM ...

WHERE ...

WITH CHECK OPTION;

Purpose

Ensures that INSERT, UPDATE, or DELETE operations do not violate the conditions used to define the view.

If an update would cause a row to no longer satisfy the view's WHERE condition, the DBMS rejects it.

Inline View

An **Inline View** is a view defined directly inside the FROM clause of an SQL query instead of being permanently created with CREATE VIEW. It exists only for the duration of that query.

Views as Authorization Mechanism

Views provide security by exposing only required data.

Example 1 – Restrict Rows

```
CREATE VIEW DEPT5EMP AS
SELECT *
FROM EMPLOYEE
WHERE Dno = 5;
```

A user with access only to this view can see employees belonging to Department 5, but cannot access records from other departments.

Example 2 – Restrict Columns

```
CREATE VIEW BASIC_EMP_DATA AS
SELECT
Fname,
Lname,
Address
FROM EMPLOYEE;
```

This view hides confidential fields such as Salary or Department and displays only basic employee information.

Advantages of Views

1. Simplifies complex SQL queries.
2. Provides data security by hiding sensitive rows and columns.
3. Reduces query complexity by avoiding repeated joins.
4. Presents customized data for different users.
5. Reflects the latest changes in base tables.

6. Improves application maintainability.
7. Supports logical data independence.

Limitations of Views

1. Many views cannot be updated.
2. Complex views may reduce performance.
3. Materialized views require maintenance.
4. Views with joins and aggregate functions are generally read-only.
5. Excessive nesting of views can make debugging difficult.

5.3 Two-Phase Locking Techniques for Concurrency Control

Introduction

In a Database Management System (DBMS), multiple users often access the database simultaneously. Several transactions may read or modify the same data items at the same time. This simultaneous execution of transactions is known as **concurrent execution**.

Although concurrency improves system performance and resource utilization, it also introduces problems such as:

- Lost Update
- Dirty Read
- Temporary Update Problem
- Incorrect Summary
- Data Inconsistency

To avoid these problems and ensure that transactions execute correctly, the DBMS uses **Concurrency Control Techniques**.

One of the most widely used concurrency control methods is the **Lock-Based Protocol**, particularly the **Two-Phase Locking (2PL) Protocol**.

Locking ensures that only authorized transactions can access a data item at a particular time, thereby maintaining database consistency and serializability.

What is Concurrency Control?

Definition

Concurrency Control is the process of managing simultaneous execution of multiple transactions so that the database remains consistent and correct.

It ensures that:

- Transactions do not interfere with one another.
- Database consistency is maintained.
- Concurrent execution produces results equivalent to some serial execution.

Need for Concurrency Control

Without concurrency control,

- Multiple transactions may update the same data simultaneously.
- One transaction may read incomplete data written by another transaction.
- Transactions may overwrite each other's updates.
- Database consistency may be lost.

Therefore, concurrency control is essential for reliable database operation.

Objectives of Concurrency Control

The primary objectives are:

- Maintain database consistency.
- Preserve data integrity.
- Ensure serializability.
- Prevent conflicting operations.
- Improve database reliability.
- Allow safe concurrent execution.
- Maximize system throughput.

Locking

Definition

A **lock** is a mechanism used by the DBMS to control access to database items.

It acts like a permission that determines whether a transaction is allowed to read or modify a data item.

Before accessing a database item, a transaction must first obtain the appropriate lock.

After completing its operation, the transaction releases the lock

Real-Life Example

Imagine a library.

If a student borrows a book, nobody else can borrow it until it is returned.

Similarly,

When a transaction locks a database item,

Other transactions may have to wait until that lock is released.

Why Locking is Needed

Suppose two bank transactions access the same account balance.

Transaction T1

Read Balance = ₹10,000

Withdraw ₹2,000

Write Balance = ₹8,000

Transaction T2

Read Balance = ₹10,000

Deposit ₹5,000

Write Balance = ₹15,000

If both execute simultaneously without locking,

Final balance may become either

₹8,000

or

₹15,000

instead of

₹13,000.

This problem occurs because transactions interfere with one another.

Locking prevents such inconsistencies.

What is a Lock?

A lock is simply a variable associated with every database item.

It stores the current access status of that item.

Whenever a transaction wants to access a data item,

It first checks the lock.

If access is allowed,

the transaction proceeds.

Otherwise,

it waits until the lock becomes available.

Types of Locks

The uploaded text discusses two major locking techniques:

1. Binary Locks
2. Shared/Exclusive (Read/Write) Locks

1. Binary Locks

Definition

A **Binary Lock** is the simplest locking mechanism.

It has only two possible states.

- Locked (1)
- Unlocked (0)

Hence it is called a Binary Lock.

States of Binary Lock

Locked (1)

The item cannot be accessed by any other transaction.

Unlocked (0)

The item is free.

Any transaction may lock and access it.

Representation

LOCK(X)

0 → Unlocked

1 → Locked

where

X = database item

Working of Binary Lock

Suppose transaction T1 wants to access item X.

Step 1

It executes

lock_item(X)

DBMS checks

LOCK(X)

Case 1

If

LOCK(X)=0

Then

LOCK(X)=1

Transaction gets the lock.

It can now read or write X.

Case 2

If

LOCK(X)=1

Another transaction already holds the lock.

Therefore,

Current transaction waits.

It remains in the waiting queue until X becomes unlocked.

After completing its work,

transaction executes

unlock_item(X)

DBMS changes

LOCK(X)=0

Now another waiting transaction may acquire the lock.

Binary Lock Algorithm

Lock Operation

lock_item(X)

If LOCK(X)=0

LOCK(X)=1

Else

Wait until LOCK(X)=0

Acquire Lock

Unlock Operation

unlock_item(X)

LOCK(X)=

Wake up one waiting transaction.

This algorithm guarantees **mutual exclusion**, meaning only one transaction can access the data item at a time.

Binary Lock Example

Assume

LOCK(Account)=0

Transaction T1

lock(Account)

LOCK(Account)=1

Read Account

Update Account

Transaction T2 arrives.

It checks

LOCK(Account)=1

Therefore,

T2 waits.

When T1 completes,

unlock(Account)

LOCK(Account)=0

Now

T2 acquires the lock.

Lock Manager

The DBMS contains a special component called the **Lock Manager**.

Its responsibilities include:

- Granting locks.
- Releasing locks.
- Maintaining waiting queues.

- Preventing unauthorized access.
- Updating the lock table.

The Lock Manager is responsible for ensuring that locking rules are correctly enforced.

Lock Table

The Lock Manager maintains a **Lock Table**.

The uploaded material describes a binary lock record with fields such as:

<Data Item Name,

LOCK,

Locking Transaction>

Waiting transactions are maintained in a queue until the lock is released.

Rules for Binary Locking

Every transaction must follow these rules:

1. Lock the item before performing any read or write operation.
2. Unlock the item after completing all operations.
3. Do not lock an item if the transaction already holds its lock.
4. Do not unlock an item unless the transaction currently holds its lock.

Following these rules ensures proper synchronization among concurrent transactions.

Limitations of Binary Locks

Although binary locking is simple,

it has major drawbacks.

Only one transaction can access the item.

Even if multiple transactions only want to read the same data,

only one transaction is allowed.

This unnecessarily reduces concurrency.

Example

Suppose

100 students are viewing their exam timetable.

All transactions only read data.

Binary locking still allows only one student at a time.

The remaining 99 students wait.

This decreases system performance.

Therefore,

Binary locking is considered **too restrictive**.

To solve this limitation,

DBMS introduces

Shared/Exclusive Locks.

2. Shared / Exclusive Locks

Also called

- Read/Write Locks
- Multiple Mode Locks

Why Shared/Exclusive Locks?

Binary locking allows only one transaction.

However,

reading does not modify data.

Therefore,

many transactions should be allowed to read simultaneously.

Only writing requires exclusive access.

To achieve this,

DBMS uses Shared/Exclusive Locks.

Types of Read/Write Locks

1. Shared Lock (Read Lock)

Symbol

S-Lock

or

Read Lock

Purpose

Allows reading only.

Multiple transactions can hold a shared lock on the same item simultaneously.

No transaction can modify the item while shared locks are active.

Example

Student Database

Student A reads marks

Student B reads marks

Student C reads marks

All three transactions can execute together.

No conflict occurs.

2. Exclusive Lock (Write Lock)

Symbol

X-Lock

or

Write Lock

Purpose

Allows both reading and writing.

Only one transaction can hold an exclusive lock.

No other transaction may read or write the item until the exclusive lock is released.

Example

T1 updates salary.

T2 wants to read salary.

T3 wants to update salary.

Both T2 and T3 must wait until T1 finishes.

States of Read/Write Lock

A database item may exist in one of three states:

Lock State	Meaning
Unlocked	Free for access
Read Locked	One or more transactions are reading
Write Locked	One transaction is writing

These three states allow higher concurrency than binary locks.

Lock Table for Shared/Exclusive Locks

The uploaded material describes a lock table containing fields such as:

<Data Item Name,

LOCK,

Number of Reads,

Locking Transaction(s)>

The table records:

- The current lock mode (read or write).
- The number of shared readers.
- The transaction(s) holding the lock.

This information helps the Lock Manager decide whether a new lock request can be granted.

Advantages of Shared/Exclusive Locks

- Better concurrency than binary locking.
- Multiple read operations can occur simultaneously.
- Ensures exclusive access during updates.
- Prevents conflicting operations.
- Maintains database consistency.
- Efficient use of system resources.

Disadvantages

- More complex than binary locking.
- Requires lock tables and lock management.
- May lead to waiting.
- Can result in deadlock if not managed properly.

Binary Lock vs Shared/Exclusive Lock

Binary Lock	**Shared/Exclusive Lock**
Two states	Three states
Locked/Unlocked	Read Locked, Write Locked, Unlocked
Only one transaction allowed	Multiple readers allowed
Very simple	More flexible
Poor concurrency	Better concurrency
Rarely used	Widely used in DBMS

5.3.1 Guaranteeing Serializability by Two-Phase Locking

A transaction may still release a lock too early and later acquire another lock. This can lead to **non-serializable schedules**, producing incorrect database results. To overcome this limitation, the **Two-Phase Locking (2PL) Protocol** is introduced.

Why is Two-Phase Locking Needed?

Problem with Ordinary Locking

Suppose two transactions follow the locking rules:

- They lock items before accessing them.
- They unlock items after use.

Even then, a transaction may unlock one item and later request a lock on another item. During this interval, another transaction may access the unlocked item, leading to interleaved execution that is **not equivalent to any serial schedule**.

Thus, **locking alone is insufficient** for ensuring serializability.

Example Scenario

Consider two transactions:

Transaction T1

- Reads data item Y.
- Unlocks Y.
- Later locks X and updates it.

Transaction T2

- Reads X.
- Unlocks X.
- Later locks Y and updates it.

Since each transaction releases one lock before acquiring another, their operations can interleave and produce incorrect results.

This type of schedule is **non-serializable** even though both transactions use locks correctly.

What is Serializability?

Definition

Serializability is a property of a transaction schedule that ensures the concurrent execution of transactions produces the **same result as some serial execution** of those transactions.

A serializable schedule guarantees:

- Correctness of data.
- Database consistency.
- No concurrency anomalies.

The primary objective of Two-Phase Locking is to ensure serializability automatically.

What is Two-Phase Locking (2PL)?

Definition

Two-Phase Locking (2PL) is a concurrency control protocol in which **all lock acquisition operations must occur before the first unlock operation**.

In other words:

- A transaction first acquires all the locks it needs.
- Only after it has finished acquiring locks does it begin releasing them.

No new lock can be acquired after the first lock is released.

Why is it Called "Two-Phase" Locking?

The transaction is divided into **two distinct phases**:

1. **Growing (Expanding) Phase**
2. **Shrinking Phase**

These two phases never overlap.

Phase 1 – Growing (Expanding) Phase

Definition

The **Growing Phase** is the first phase of the transaction during which:

- The transaction **can acquire new locks**.
- It **cannot release any lock**.

Once the transaction enters this phase, it continues acquiring locks until all required locks have been obtained.

Characteristics

- New locks are permitted.
- Unlocking is not allowed.
- Transaction prepares to execute safely.
- Database consistency is protected.

Example

Transaction T1 requires locks on:

- Student Table
- Marks Table
- Result Table

Growing phase:

Read_Lock(Student)

Write_Lock(Marks)

Write_Lock(Result)

No lock is released in this phase.

Phase 2 – Shrinking Phase

Definition

The **Shrinking Phase** begins when the transaction releases its **first lock**.

During this phase:

- Locks can only be released.
- No new lock can be acquired.

Characteristics

- Unlocking begins.
- Lock acquisition is prohibited.

- Transaction gradually frees resources.

Example

Unlock(Student)

Unlock(Marks)

Unlock(Result)

No further Read_Lock() or Write_Lock() operation is allowed after the first Unlock().

Important Rule of Two-Phase Locking

A transaction **must never acquire a new lock after releasing its first lock**.

This single rule guarantees that all schedules generated under 2PL are serializable.

Diagram of Two-Phase Locking

Two-Phase Locking

Growing Phase	Shrinking Phase
Acquire Lock ✓	Acquire Lock ✗
Acquire Lock ✓	Release Lock ✓
Acquire Lock ✓	Release Lock ✓
Release Lock ✗	Release Lock ✓

The transition from the Growing Phase to the Shrinking Phase occurs at the **first unlock operation**

Lock Point

Definition

The **Lock Point** is the instant at which a transaction has acquired **all the locks it will ever need**.

After the lock point:

- No new locks can be obtained.
- The transaction enters the shrinking phase.

The lock point plays a crucial role in proving serializability under 2PL.

Example of Two-Phase Locking

Consider the following transaction:

Read_Lock(A

Read(A)

Write_Lock(B)

Read(B)

Write(B)

Unlock(A)Unlock(B)

Growing Phase

Read_Lock(A)

Write_Lock(B)

Shrinking Phase

Unlock(A)

Unlock(B)

The transaction follows the Two-Phase Locking protocol because **all locks are acquired before any lock is released**.

Example of a Transaction Violating 2PL

Read_Lock(A)

Read(A)

Unlock(A)

Write_Lock(B)

Write(B)

This transaction **does not follow 2PL** because it acquires a new lock (Write_Lock(B)) after releasing A.

Such execution may produce a non-serializable schedule and is therefore not permitted under the Two-Phase Locking protocol.

How Two-Phase Locking Guarantees Serializability

Under 2PL:

- Transactions obtain all necessary locks before releasing any.
- Other transactions cannot interfere with partially completed work.
- Conflicting operations are properly ordered.
- Every schedule generated by 2PL is equivalent to some serial schedule.

Thus, 2PL **guarantees serializability without requiring separate serializability testing**.

Limitations of Two-Phase Locking

Although Two-Phase Locking guarantees serializability, it has certain drawbacks.

1. Reduced Concurrency

A transaction may hold a lock longer than necessary.

Even after completing operations on a data item, it cannot release the lock if it still needs to acquire another lock later.

As a result:

- Other transactions must wait.
- Parallel execution decreases.

2. Waiting Time Increases

Transactions often wait for locks held by others, reducing throughput.

3. Possibility of Deadlock

Two or more transactions may wait indefinitely for each other's locks, leading to **deadlock**.

Lock Conversion

Sometimes a transaction needs to change the type of lock it holds on a data item.

This process is known as **Lock Conversion**.

There are two forms:

1. Upgrading
2. Downgrading

Upgrading of Locks

Definition

Upgrading is the process of converting a **Shared (Read) Lock** into an **Exclusive (Write) Lock**.

Why is Upgrading Needed?

A transaction may initially read a data item.

Later, it decides to modify the same item.

Instead of releasing the shared lock and requesting a new exclusive lock, the existing lock is upgraded.

Example

Initially:

Read_Lock(Student)

Later:

Write_Lock(Student)

The shared lock is upgraded to an exclusive lock.

Condition for Upgrading

Upgrading is allowed **only if no other transaction currently holds a shared lock on that data item**.

Otherwise, the transaction must wait until all other shared locks are released.

Downgrading of Locks

Definition

Downgrading is the process of converting an **Exclusive (Write) Lock** into a **Shared (Read) Lock**.

Why is Downgrading Needed?

A transaction may complete all update operations but still need to read the same data.

Instead of holding the exclusive lock unnecessarily, it converts it into a shared lock, allowing other transactions to read simultaneously.

Example

Initially:

Write_Lock(Result)

After updates:

Read_Lock(Result)

The write lock is downgraded to a read lock.

Condition for Downgrading

Downgrading is performed after the transaction has completed its write operations.

It allows better concurrency by permitting multiple read operations

Lock Conversion Rules in 2PL

When lock conversion is permitted:

- **Upgrading** must occur only during the **Growing Phase**.
- **Downgrading** must occur only during the **Shrinking Phase**.

These restrictions preserve the correctness of the Two-Phase Locking protocol.

Advantages of Two-Phase Locking

- Guarantees serializable schedules.
- Maintains database consistency.

- Prevents conflicting operations.
- Simple and widely implemented.
- Forms the basis for many commercial DBMS concurrency control mechanisms.

Disadvantages of Two-Phase Locking

- May reduce concurrency because locks are held longer.
- Can cause transactions to wait.
- Deadlocks are possible.
- Additional overhead for lock management.

Variations of Two-Phase Locking (2PL)

The basic Two-Phase Locking protocol guarantees **serializability**, but it does not solve every concurrency problem. For example, transactions may still enter a deadlock, and recovery after transaction failure may become difficult.

To overcome these limitations, several improved versions of Two-Phase Locking have been developed. The uploaded material discusses four important variants:

1. Basic Two-Phase Locking (Basic 2PL)
2. Conservative Two-Phase Locking (Static 2PL)
3. Strict Two-Phase Locking (Strict 2PL)
4. Rigorous Two-Phase Locking (Rigorous 2PL)

1. Basic Two-Phase Locking (Basic 2PL)

Definition

Basic Two-Phase Locking is the standard implementation of the Two-Phase Locking protocol.

A transaction follows two phases:

- Growing Phase
- Shrinking Phase

All locks must be acquired before the first unlock operation.

Working

The transaction performs the following sequence:

Acquire Lock(A)

Acquire Lock(B)

Acquire Lock(C)

Lock Point

Unlock(A)

Unlock(B)

Unlock(C)

After the first unlock operation, the transaction cannot request any new lock.

Characteristics

- Guarantees serializability.
- Simple to understand.
- Widely used as the foundation of other locking protocols.
- Deadlocks are possible.

Advantages

- Ensures correct transaction execution.
- Easy to implement.
- Prevents conflicting operations.

Disadvantages

- Deadlock may occur.
- Locks may be held longer than necessary.
- Reduced concurrency.

2. Conservative Two-Phase Locking (Static 2PL)

Definition

Conservative Two-Phase Locking is a variation in which a transaction must acquire **all required locks before it starts execution**.

If any one of the required locks is unavailable, the transaction acquires **none of the locks** and waits until all required locks become available.

Why is it Called Static 2PL?

The transaction already knows:

- Which items it will read.
- Which items it will write.

These are called:

- **Read Set** – all items the transaction reads.
- **Write Set** – all items the transaction writes.

The transaction declares these sets before execution begins.

Working

Suppose Transaction T needs

- Student
- Faculty
- Marks

Before starting,

DBMS attempts to lock

Student

Faculty

Marks

Case 1

All locks available

↓

Transaction begins.

Case 2

Faculty locked

↓

Transaction waits.

No locks are granted.

Only after all three become available does execution begin.

Characteristics

- All locks acquired before execution.
- No partial locking.
- Deadlock cannot occur.
- Requires advance knowledge of Read Set and Write Set.

Advantages

- Completely deadlock-free.
- Safe execution.
- No waiting after transaction starts.

Disadvantages

- Difficult to know all required items in advance.
- Poor concurrency.
- Locks may remain unused for a long period.

Because of these practical limitations, Conservative 2PL is rarely used in commercial DBMSs.

3. Strict Two-Phase Locking (Strict 2PL)

Definition

In **Strict Two-Phase Locking**, a transaction does **not release any Exclusive (Write) Lock until it commits or aborts**.

Read locks may be released earlier, but write locks are held until the transaction finishes.

Why Strict 2PL?

Suppose

Transaction T1 modifies account balance.

Before committing,

Transaction T2 reads that modified balance.

If T1 later aborts,

T2 has already read invalid data.

This problem is called a **Dirty Read**.

Strict 2PL prevents this situation.

Working

Transaction

Write_Lock(Account)

Update Balance

Commit

Unlock(Account)

Notice

The lock is released only **after Commit**

Characteristics

- Write locks held until Commit or Abort.
- Prevents dirty reads.
- Produces strict schedules.
- Deadlock still possible.

Advantages

- Easy recovery.

- Prevents cascading rollback.
- Guarantees recoverable schedules.
- Most commonly implemented in commercial DBMS.

Disadvantages

- Deadlocks may occur.
- Transactions wait longer.
- Reduced concurrency.

4. Rigorous Two-Phase Locking (Rigorous 2PL)

Definition

Rigorous Two-Phase Locking is the most restrictive form of Two-Phase Locking.

A transaction releases **neither Read Locks nor Write Locks until Commit or Abort**.

All locks remain held until the transaction terminates.

Working

Read_Lock(Student)

Write_Lock(Result)

Commit

Unlock(Student)

Unlock(Result)

Every lock remains held until commit.

Characteristics

- Read locks held until commit.
- Write locks held until commit.
- Simplest implementation.
- Very safe.
- Lowest concurrency.

Advantages

- Guarantees strict schedules.
- Simplifies recovery.
- Very easy lock management.
- Highly reliable.

Disadvantages

- Maximum waiting time.
- Lowest concurrency.
- Deadlocks still possible.

Comparison of All Four Variants

Feature	Basic 2PL	Conservative 2PL	Strict 2PL	Rigorous 2PL
Guarantees Serializability	✓	✓	✓	✓
Deadlock Possible	✓	✗	✓	✓
Lock All Items Before Start	✗	✓	✗	✗
Hold Write Locks Until Commit	✗	✗	✓	✓
Hold Read Locks Until Commit	✗	✗	✗	✓
Concurrency	Medium	Low	Medium	Lowest
Practical Usage	Moderate	Rare	Very High	Moderate

Deadlock

Definition

A **Deadlock** is a situation in which two or more transactions wait indefinitely for each other to release locks.

Since every transaction is waiting,

none of them can proceed.

Thus, all transactions become permanently blocked.

Example of Deadlock

Suppose

Transaction T1

Locks X

Needs Y

Transaction T2

Locks Y

Needs X

Now

T1 → waiting for Y

T2 → waiting for X

Neither transaction can continue.

This is called **Deadlock**.

Diagram

T1

Lock(X)

↓

Waiting for Y

T2

Lock(Y)

↓

Waiting for X

Both wait forever.

Why Deadlock Occurs

Deadlock occurs when

- Transactions hold some locks.
- They request additional locks.
- Requested locks are held by other waiting transactions.

This creates a circular waiting condition.

Problems Caused by Deadlock

- Transactions never finish.
- Resources remain occupied.
- System throughput decreases.
- Database performance suffers.

Deadlock Prevention

Deadlock prevention techniques ensure that deadlock never occurs.

The uploaded material discusses several methods.

1. Conservative 2PL

Acquire every required lock before execution.

Since transactions never wait after execution begins,

deadlock cannot occur.

2. Resource Ordering

Assign an order to database items.

Example

↓

B

↓

C

↓

D

Every transaction must request locks only in this order.

Circular waiting becomes impossible.

3. Wait-Die Scheme

Uses transaction timestamps.

Rule

Older transaction

↓

Allowed to wait.

Younger transaction

↓

Aborted (dies).

After abort,

it restarts with the same timestamp.

Example

T1 started first.

T2 started later.

If

T2 requests resource held by T1,

↓

T2 dies.

If

T1 requests resource held by T2,

↓

T1 waits.

This prevents circular waiting.

4. Wound-Wait Scheme

Opposite of Wait-Die.

Rule

Older transaction

↓

Aborts younger transaction.

Younger transaction

↓

Allowed to wait.

The older transaction "wounds" the younger one, forcing it to restart later.

5. No-Wait Algorithm

If a transaction cannot obtain a lock immediately,

it is aborted instantly.

Since no transaction waits,

deadlock cannot occur.

6. Cautious Waiting

A transaction waits only if the transaction holding the lock is **not already waiting**.

Otherwise,

the requesting transaction is aborted.

This avoids circular waiting and prevents deadlock.

Deadlock Detection

Instead of preventing deadlocks,

the DBMS may allow them to occur and then detect them.

The uploaded material describes the **Wait-for Graph** method.

Wait-for Graph

Definition

A **Wait-for Graph (WFG)** is a directed graph used to detect deadlocks.

- Each node represents a transaction.
- A directed edge **Ti → Tj** means **Ti is waiting for Tj**.

Rule

If the Wait-for Graph contains a **cycle**,

then a deadlock exists.

If no cycle exists,

there is no deadlock

Example

T1 → T2

↑ ↓

T4 ← T3

Cycle exists.

Therefore,

Deadlock exists

Victim Selection

After detecting a deadlock,

one transaction must be aborted.

The aborted transaction is called the **Victim**.

The DBMS generally selects:

- Younger transaction.
- Transaction with fewer updates.
- Transaction with lower execution cost.

After aborting the victim, the remaining transactions continue normally.

Timeout Method

Instead of constructing a Wait-for Graph,

the DBMS may use a **Timeout**.

If a transaction waits longer than a predefined time,

the DBMS assumes a deadlock and aborts the transaction.

Advantages

- Very simple.
- Low overhead.
- Easy implementation.

Disadvantages

- May abort transactions even when no deadlock exists.

Starvation

Definition

Starvation occurs when a transaction waits indefinitely because other transactions continuously receive priority.

Unlike deadlock,

the system continues working,

but one transaction never gets a chance to execute.

Causes of Starvation

- Unfair scheduling.

- Repeated victim selection.
- Continuous arrival of higher-priority transactions.

Prevention of Starvation

The uploaded material suggests:

- **First-Come-First-Served (FCFS)** waiting queues to ensure fairness.
- **Priority Aging**, where the priority of a waiting transaction increases over time until it eventually executes.
- Avoid repeatedly selecting the same transaction as the victim during deadlock resolution.

Deadlock vs Starvation

Deadlock	Starvation
Transactions wait for each other	One transaction waits indefinitely
System becomes blocked	System continues executing
Caused by circular waiting	Caused by unfair scheduling
Requires detection/prevention	Requires fair scheduling
All involved transactions stop	Only one transaction suffers

5.4 Concurrency Control Based on Timestamp Ordering

Introduction

In the previous chapter, we studied **Two-Phase Locking (2PL)**, which guarantees serializability using locks. Although locking is effective, it has certain disadvantages:

- Transactions may have to wait for locked data items.
- Deadlocks may occur.
- Transactions may need to be aborted and restarted because of deadlocks.

To overcome these problems, another concurrency control technique called **Timestamp Ordering (TO)** is used.

Unlike locking protocols, Timestamp Ordering **does not use locks**. Instead, it uses **transaction timestamps** to determine the order in which transactions should execute.

The main objective of Timestamp Ordering is to ensure that all conflicting operations occur according to the chronological order of transaction timestamps, thereby guaranteeing serializability without using locks.

Why Timestamp Ordering is Needed?

Consider two transactions:

- Transaction T1 starts first.
- Transaction T2 starts later.

Since T1 started earlier, it should logically execute before T2.

Timestamp Ordering ensures that this order is always maintained, even if the actual operations of the two transactions are interleaved.

Because no locks are used:

- Transactions do not wait for locks.
- Deadlocks cannot occur.
- Execution becomes more efficient in systems with many concurrent transactions.

Difference Between Two-Phase Locking and Timestamp Ordering

Two-Phase Locking (2PL)	Timestamp Ordering (TO)
Uses locks	Uses timestamps
Transactions may wait	Transactions never wait for locks
Deadlock may occur	Deadlock cannot occur
Serializability through locking protocol	Serializability through timestamp order
Lock manager required	Timestamp manager required

What is a Timestamp?

Definition

A **Timestamp** is a unique identifier assigned by the Database Management System (DBMS) to every transaction.

It indicates the order in which transactions enter the system.

Usually, the transaction that starts first receives the **smallest timestamp**, and transactions that start later receive larger timestamp values.

The timestamp of transaction **T** is represented as:

TS(T)

Example

Suppose three transactions enter the system.

Transaction	Timestamp
T1	1
T2	2
T3	3

Here,

TS(T1) < TS(T2) < TS(T3)

Therefore,

T1 is the oldest transaction.

T3 is the youngest transaction.

Characteristics of Timestamps

A timestamp has the following properties:

- Every transaction receives a unique timestamp.
- No two transactions have the same timestamp.
- Timestamps determine transaction execution order.

- Smaller timestamp means older transaction.
- Larger timestamp means younger transaction.

How are Timestamps Generated?

The uploaded material describes two methods

1. Counter Method

A counter is maintained by the DBMS.

Whenever a new transaction begins,

the counter increases by one.

Example

Counter = 1

↓

T1 gets TS = 1

Counter = 2

↓

T2 gets TS = 2

Counter = 3

↓

T3 gets TS = 3

If the counter reaches its maximum value,

the DBMS resets it after no transactions are executing.

2. System Clock Method

Instead of using a counter,

the DBMS uses the current system date and time.

Example

10:15:20

↓

Transaction T1

10:15:25

↓

Transaction T2

Since no two transactions are assigned timestamps during the same clock tick,

every transaction receives a unique timestamp.

Timestamp Ordering (TO) Algorithm

Definition

Timestamp Ordering (TO) is a concurrency control protocol that ensures transactions execute according to the order of their timestamps.

It guarantees that the resulting schedule is equivalent to the serial schedule defined by timestamp order.

Unlike Two-Phase Locking,

transactions do not wait for locks.

Instead,

the DBMS checks whether every read and write operation follows timestamp order.

If an operation violates timestamp order,

the corresponding transaction is aborted and restarted.

Principle of Timestamp Ordering

Suppose

TS(T1)=10

TS(T2)=20

Since

10 < 20

T1 is older.

Therefore,

all conflicting operations must appear as though T1 executed before T2.

If T2 somehow performs an operation that violates this order,

the DBMS rejects the operation

Timestamp Values Associated with Each Data Item

For every database item **X**, the DBMS maintains two timestamps.

1. Read Timestamp

Definition

Read Timestamp, represented as

read_TS(X)

is the largest timestamp among all transactions that have successfully **read** data item X.

In simple words,

it stores the timestamp of the youngest transaction that has read X.

Example

Suppose

T1 reads X

TS=5

T2 reads X

TS=8

T3 reads X

TS=10

Then

read_TS(X)=10

because T3 is the latest successful reader.

2. Write Timestamp

Definition

Write Timestamp, represented as

write_TS(X)

is the largest timestamp among all transactions that have successfully **written** data item X.

It stores the timestamp of the youngest transaction that has modified X.

Example

T1 writes X

TS=6

T2 writes X

TS=15

Then

write_TS(X)=15

Basic Timestamp Ordering Algorithm

The Basic Timestamp Ordering Algorithm checks every **Read** and **Write** request.

If the operation follows timestamp order,

it is executed.

Otherwise,

the transaction is aborted and restarted

Case 1 – Write Operation

Suppose transaction **T** wants to perform

write_item(X)

The DBMS performs the following checks

Condition 1

If

read_TS(X) > TS(T)

Then

Abort transaction T.

Why?

A younger transaction has already read X.

Allowing the older transaction to write now would violate timestamp ordering.

Condition 2

If

write_TS(X) > TS(T)

Abort transaction T.

Why?

A younger transaction has already updated X.

Allowing the older transaction to overwrite the value would violate timestamp ordering

Otherwise

If neither condition is true,

The write operation is allowed.

After successful writing,

write_TS(X)=TS(T)

Flowchart for Write Operation

Transaction wants Write(X)

↓

Is read_TS(X)>TS(T)

↓]

Yes → Abort

↓

N

↓

Is write_TS(X)>TS(T)?

↓

Yes → Abort

↓

No

↓

Write

↓

Update write_TS(X)

Case 2 – Read Operation

Suppose transaction T performs

read_item(X)

The DBMS checks

write_TS(X)>TS(T)?

If True

Abort transaction T.

Reason

A younger transaction has already written X.

Reading now would violate timestamp ordering

Otherwise

Read operation is allowed.

After reading,

read_TS(X

Maximum

(Current read_TS(X)

TS(T))

Flowchart for Read Operation

Transaction wants Read(X)

↓

Is write_TS(X)>TS(T)?

↓

Yes

↓

Abort Transaction

↓

No

↓

Read X

↓

Update read_TS(X)

Cascading Rollback

Definition

If one transaction is aborted,

all transactions that used the values written by it must also be rolled back.

This chain reaction is called **Cascading Rollback**.

Example

T1 writes X

↓

T2 reads X

↓

T3 reads X

↓

T1 aborts

Now

T2 must rollback.

T3 must also rollback.

Thus,

one abort causes multiple rollbacks.

This is the major drawback of the Basic Timestamp Ordering Algorithm.

Advantages of Basic Timestamp Ordering

- Guarantees conflict serializability.
- No locking required.
- Deadlocks cannot occur.
- Simple timestamp comparison.

Disadvantages

- Frequent transaction aborts.
- Cascading rollback possible.
- Starvation may occur due to repeated restarts.
- Recoverability is not guaranteed.

Strict Timestamp Ordering (Strict TO)

Definition

Strict Timestamp Ordering is an improved version of Basic TO.

In this protocol,

if a transaction wants to read or write a data item that has been written by another transaction,

it must wait until the writing transaction commits or aborts.

Purpose

Strict TO eliminates

- Dirty reads
- Cascading rollback

Thus,

it produces **strict schedules** that simplify recovery

Working

Suppose

T1 writes X

↓

T2 wants to read X

If T1 has not yet committed,

T2 waits.

Only after T1 commits or aborts can T2 proceed.

Advantages

- Recoverable schedules.
- Prevents cascading rollback.
- Guarantees conflict serializability.
- No deadlocks.

Thomas's Write Rule

Definition

Thomas's Write Rule is a modification of the Basic Timestamp Ordering Algorithm.

It reduces unnecessary transaction aborts by ignoring obsolete write operations.

Working

Suppose transaction T wants to write X.

Three cases are considered.

Case 1

If

read_TS(X)>TS(T)

Abort transaction.

Reason:

A younger transaction has already read X.

Case 2

If

write_TS(X)>TS(T)

Do **not abort**.

Simply ignore the write operation.

Reason:

A younger transaction has already written a newer value.

The current write has become obsolete.

Case 3

Otherwise,

Perform the write operation.

Update

write_TS(X)=TS(T)

Why Thomas's Write Rule is Better?

Instead of aborting transactions unnecessarily,

obsolete writes are simply ignored.

Therefore,

- Fewer transaction restarts.
- Better system performance.
- Improved concurrency.

Comparison of Basic TO, Strict TO and Thomas's Write Rule

Feature	Basic TO	Strict TO	Thomas's Write Rule
Uses Locks	No	No	No
Uses Timestamps	Yes	Yes	Yes
Deadlock	Impossible	Impossible	Impossible
Cascading Rollback	Possible	Prevented	Prevented
Recoverability	Not Guaranteed	Guaranteed	Better than Basic TO
Obsolete Writes	Abort Transaction	Wait	Ignore Write

Advantages of Timestamp Ordering

- No locking mechanism required.
- Deadlocks never occur.
- Ensures conflict serializability.
- Simple timestamp comparison.
- Suitable for highly concurrent systems.

Disadvantages of Timestamp Ordering

- Transactions may be aborted frequently.
- Cascading rollback may occur in Basic TO.
- Starvation is possible due to repeated restarts.

- Timestamp maintenance introduces overhead.

5.5 Multiversion Concurrency Control Techniques (MVCC)

Introduction

In the previous chapters, we studied two important concurrency control techniques:

- Two-Phase Locking (2PL)
- Timestamp Ordering (TO)

Although both techniques ensure serializability, they have certain disadvantages.

- Locking may cause **deadlocks**.
- Timestamp Ordering may cause **frequent transaction aborts**.
- Read operations may be rejected because of conflicts with write operations.

To overcome these problems, Database Management Systems use another advanced concurrency control technique called **Multiversion Concurrency Control (MVCC)**.

Unlike previous techniques, MVCC maintains **multiple versions of the same database item** instead of storing only one copy.

This enables multiple transactions to access different versions of the same data item simultaneously, thereby increasing concurrency and reducing unnecessary waiting

What is Multiversion Concurrency Control?

Definition

Multiversion Concurrency Control (MVCC) is a concurrency control technique in which the database maintains **multiple versions (copies)** of each data item.

Whenever a transaction updates a data item:

- A **new version** of the item is created.
- The **old version is retained** instead of being overwritten.

When another transaction requests the same item, the DBMS selects the most appropriate version to maintain serializability.

Basic Idea of MVCC

Instead of replacing old data,

MVCC stores

Old Version

↓

New Version

↓

Latest Version

Every transaction reads the version that is appropriate according to the concurrency control algorithm.

Why is MVCC Needed?

Suppose

Transaction T1 updates Account Balance.

While T1 is updating,

Transaction T2 only wants to read the balance.

In Two-Phase Locking,

T2 may have to wait.

In Timestamp Ordering,

T2 may be aborted.

However,

in MVCC,

T2 simply reads the previous committed version of the balance.

Therefore,

both transactions proceed simultaneously.

Working Principle

Whenever a transaction performs

Write(X)

DBMS

↓

Creates a **new version** of X.

Old version remains available.

When another transaction performs

Read(X)

DBMS chooses

↓

The correct version of X.

Thus,

reading and writing can occur simultaneously.

Example

Initially

Salary = ₹50,000

Transaction T1 updates salary.

Instead of replacing

₹50,000

DBMS stores

Version 1

Salary = ₹50,000

Version 2

Salary = ₹60,000

Old transactions continue reading

₹50,000.

New transactions read

₹60,000

Advantages of Multiple Versions

Maintaining multiple versions provides several benefits.

1. Read Operations Rarely Wait

Readers can access an older committed version even while another transaction updates the latest version.

2. Higher Concurrency

Many transactions can execute simultaneously

3. Better Performance

Since read operations do not wait,

overall throughput improves

4. Improved User Experience

Users experience fewer delays because read operations are almost always successful.

Drawback of MVCC

The major disadvantage is **storage overhead**.

Since multiple versions of every data item are stored,

additional memory is required.

Older versions occupy storage until they are removed.

When is MVCC Useful?

MVCC is especially useful when:

- Many users only read data.
- Few users modify data.
- Historical versions must be preserved.
- Recovery requires old values.

Historical Databases

Some applications intentionally preserve all previous versions.

Example

Bank transactions

Medical records

Income tax records

Audit logs

These systems are called **Temporal Databases** because they maintain the history of every change.

Since old versions already exist,

MVCC introduces almost no additional storage cost.

Types of Multiversion Concurrency Control

The uploaded material discusses two important techniques.

1. Multiversion Timestamp Ordering
2. Multiversion Two-Phase Locking (MV2PL)

5.5.1 Multiversion Timestamp Ordering

Introduction

This technique combines

- Timestamp Ordering
- Multiple Versions

Instead of maintaining one copy,

DBMS stores

X1

X2

X3

...

Xn

Every version has its own timestamps.

Versions of Data Item

Suppose

Database Item

Account

Versions become

Account1

Account2

Account

...

Each version stores

- Data value
- Read Timestamp
- Write Timestamp

Timestamps Associated with Each Version

Every version Xi maintains two timestamps.

1. Read Timestamp

Definition

The **Read Timestamp** of version Xi is the largest timestamp of all transactions that have successfully read Xi.

Notation

read_TS(Xi)

Example

T1 reads Version 1

TS=5

T2 reads Version 1

TS=9

Then

read_TS(X1)=9

2. Write Timestamp

Definition

The **Write Timestamp** of version Xi is the timestamp of the transaction that created Xi.

Notation

write_TS(Xi)

Example

T5 creates Version X3

Then

write_TS(X3)=5

Creation of New Version

Whenever transaction T performs

Write(X)

DBMS creates

New Version

X(k+1)

Both timestamps become

read_TS(X(k+1))

=

TS(T)

---

write_TS(X(k+1))

=

TS(T)

Read Operation Algorithm

Suppose transaction T wants

Read(X)

DBMS performs

Step 1

Find the newest version whose

write_TS

≤

TS(T)

Step 2

Return that version.

Step 3

Update

read_TS

if required.

Why?

Older transactions should never read data written by younger transactions.

Hence,

the newest acceptable version is selected.

Example

Versions

X1

write_TS=5

X2

write_TS=12

X3

write_TS=20

Transaction

TS=15

Can read

X2

because

12 ≤ 15

but

cannot read

X3

because

20 > 15.

Write Operation Algorithm

Suppose transaction T wants

Write(X)

DBMS first identifies

the latest version

whose

write_TS

≤

TS(T)

Then

checks

read_TS(Xi)

>

TS(T)

If True

Abort transaction.

Reason

A younger transaction has already read that version.

Writing now would violate timestamp ordering.

Otherwise

Create

New Version

Xj

with

read_TS

=

TS(T)

---

write_TS

=

TS(T)

Important Observation

Read Operations

Almost always succeed.

They simply select the appropriate version.

Write Operations

May fail.

If timestamp ordering is violated,

transaction is rolled back.

Cascading Rollback

One drawback remains.

Suppose

T1

↓

writes Version

↓

T2 reads it

↓

T1 aborts

Then

T2 must also rollback.

Thus,

cascading rollback is still possible.

To avoid this,

transactions should commit only after the transactions whose versions they read have committed.

Advantages of Multiversion Timestamp Ordering

- High concurrency.
- Read operations rarely fail.
- Deadlocks do not occur.

- Older versions remain available.
- Better performance than basic timestamp ordering.

Disadvantages

- Requires more storage.
- Multiple versions must be maintained.
- Cascading rollback may still occur.
- More complex implementation.

5.5.2 Multiversion Two-Phase Locking (MV2PL)

Introduction

This technique combines

- Multiple Versions
- Two-Phase Locking

Unlike standard Two-Phase Locking,

it introduces an additional lock.

Lock Modes

Instead of two lock modes,

MV2PL has

Three lock modes.

Read Lock

↓

Write Lock

↓

Certify Lock

What is Certify Lock?

Definition

A **Certify Lock** is a special lock obtained by a transaction **before committing**.

It confirms that the updated version is ready to become the committed version.

Lock States

A data item can be

Unlocked

Read Locked

Write Locked

Certify Locked

Working of MV2PL

Suppose

Transaction T wants to update X.

Step 1

Acquire

Write Lock

Step 2

Create

Local Version

X'

instead of modifying

Committed Version X.

During Update

Other transactions continue reading

Committed Version

X

while

T modifies

Local Version

X'

Thus,

Readers are never blocked.

Before Commit

Transaction requests

Certify Lock

Only after obtaining the Certify Lock

can

Local Version

↓

becomes

Committed Version

Old local version is discarded.

Why Certify Lock?

Without the Certify Lock,

another transaction might read inconsistent data during commit.

Therefore,

Certify Lock guarantees safe replacement of the committed version.

Advantages of MV2PL

- Read operations continue during writes.
- Better concurrency than standard 2PL.
- Prevents cascading aborts because readers access only committed versions.
- Suitable for read-intensive databases.

Disadvantages

- Deadlocks are still possible.
- More complicated implementation.
- Additional storage required.
- Certify Lock may delay commit.

Standard 2PL vs MV2PL

Standard 2PL	MV2PL
Two locks	Three locks
Read Lock + Write Lock	Read Lock + Write Lock + Certify Lock
Readers blocked during writes	Readers continue reading committed version
One version	Multiple versions
Lower concurrency	Higher concurrency

Multiversion Timestamp Ordering vs Multiversion 2PL

MV Timestamp Ordering	MV2PL
Uses timestamps	Uses locks
No deadlocks	Deadlocks possible
Reads choose correct version	Reads access committed version
Writes create new version	Writes create local version
May cause cascading rollback	Prevents cascading aborts

Advantages of Multiversion Concurrency Control

- High degree of concurrency.
- Readers rarely wait.

- Better performance.
- Older versions available for recovery.
- Suitable for historical databases.
- Improves throughput.

Disadvantages

- Requires additional storage.
- Complex version management.
- Garbage collection of obsolete versions required.
- More difficult implementation than single-version techniques.

5.6 Validation (Optimistic) Techniques and Snapshot Isolation Concurrency Control

Introduction

In the previous chapters, we studied various concurrency control techniques such as:

- Two-Phase Locking (2PL)
- Timestamp Ordering (TO)
- Multiversion Concurrency Control (MVCC)

In all these techniques, the DBMS performs **checking before executing a database operation**.

For example:

- In **Locking**, the DBMS checks whether the required data item is already locked.
- In **Timestamp Ordering**, the DBMS checks the transaction timestamp against the read and write timestamps of the data item.

These repeated checks introduce **execution overhead**, slowing down transaction processing.

To reduce this overhead, another technique called **Validation-Based Concurrency Control** or **Optimistic Concurrency Control (OCC)** is used.

Unlike previous methods, **Optimistic Concurrency Control performs no checking while the transaction is executing**. Instead, all checking is postponed until the transaction reaches its end.

Why Optimistic Concurrency Control?

Consider a university database where:

- Thousands of students are viewing their examination results.
- Only a few administrators are updating the marks.

In such a situation:

- Most transactions only read data.
- Very few transactions conflict.

Performing lock checks for every read operation wastes system resources.

Instead,

the DBMS optimistically assumes that **conflicts are rare** and performs validation only after the transaction finishes execution.

Hence the name **Optimistic Concurrency Control**.

What is Validation-Based Concurrency Control?

Definition

Validation-Based Concurrency Control is a concurrency control technique in which:

- Transactions execute without acquiring locks.
- Updates are made only to **local copies** of data.
- At the end of execution, the transaction is validated.
- If validation succeeds, updates are copied to the database.
- Otherwise, the transaction is aborted and restarted.

This technique is also called:

- **Optimistic Concurrency Control (OCC)**
- **Validation Technique**

- **Certification Technique**

Basic Principle

Instead of checking every database operation,

the DBMS follows the sequence:

Transaction Executes

↓

Validation

↓

If Successful

↓

Commit

↓

Otherwise

↓

Abort and Restart

Thus,

checking occurs only once,

at the end of the transaction.

Why is it Called "Optimistic"?

The technique assumes that

- Conflicts between transactions are very rare.
- Most transactions will complete successfully.
- Therefore, expensive checking during execution is unnecessary.

This optimistic assumption works well for systems where read operations greatly outnumber write operations.

Local Copies (Transaction Workspace)

One important feature of Optimistic Concurrency Control is the use of **local copies**.

Whenever a transaction modifies data,

the original database is **not updated immediately**.

Instead,

the transaction stores all updates in its own private workspace.

Example

Database

Salary = ₹50,000

↓

Transaction Workspace

Salary = ₹60,000

The original database remains unchanged until validation succeeds.

Phases of Validation-Based Concurrency Control

Optimistic Concurrency Control consists of **three phases**.

1. Read Phase
2. Validation Phase
3. Write Phase

These phases are performed sequentially.

Phase 1 – Read Phase

Definition

During the Read Phase,

the transaction:

- Reads committed data from the database.
- Performs calculations.

- Stores updates only in its local workspace.

No changes are made to the actual database.

Activities

- Read committed values.
- Execute transaction logic.
- Store modifications locally.
- No locking.
- No validation.

Example

Suppose

Balance = ₹20,000

Transaction

Withdraw ₹5,000

Workspace

Balance = ₹15,000

Database still contains

₹20,000

Phase 2 – Validation Phase

Definition

The Validation Phase checks whether committing the transaction would violate serializability.

This is the most important phase of Optimistic Concurrency Control.

If validation succeeds,

the transaction proceeds to the Write Phase.

Otherwise,

it is aborted.

Purpose

The DBMS checks whether

- Another transaction has modified the same data.
- Conflicts have occurred.
- Serializability will be maintained.

Only if all conditions are satisfied is the transaction considered valid.

Phase 3 – Write Phase

Definition

During the Write Phase,

the updates stored in the transaction workspace are copied to the actual database.

If validation fails,

this phase is skipped.

Instead,

the transaction is restarted

Advantages of Optimistic Concurrency Control

1. No Locking Overhead

Transactions execute without acquiring locks

2. High Performance

No checking occurs during execution.

3. No Deadlocks

Since locks are not used,

deadlocks cannot occur.

4. Better Concurrency

Transactions execute independently.

5. Suitable for Read-Intensive Systems

Ideal where conflicts are rare.

Disadvantages

1. Transactions May Restart

If validation fails,

the entire transaction must be executed again.

2. Poor Performance Under Heavy Contention

If many transactions update the same data,

many validations fail.

3. Wasted Computation

A transaction may execute completely,

only to be aborted during validation.

Validation Information Maintained by DBMS

The DBMS maintains:

- Read Set
- Write Set
- Transaction timestamps
- Start time of Read Phase
- End time of Read Phase
- Start time of Validation Phase
- End time of Write Phase

This information is required for conflict checking.

Read Set

Definition

The **Read Set** is the collection of all data items read by a transaction.

Example

Transaction

Reads

Student

Marks

Result

Read Set

{Student, Marks, Result}

Write Set

Definition

The **Write Set** is the collection of all data items updated by the transaction.

Example

Updates

Marks

Attendance

Write Set

{Marks, Attendance}

Validation Conditions

Suppose

Transaction **Ti** is being validated.

The DBMS compares it with another transaction **Tj**.

Validation succeeds if **any one** of the following conditions is true.

Condition 1

Transaction Tj completes its Write Phase **before Ti starts its Read Phase**.

Diagram

Tj

Write Complete

↓

Ti

Read Starts

Since Tj has already finished,

there is no conflict.

Condition 2

Ti starts its Write Phase **after Tj completes its Write Phase**,

and

Ti's Read Set has **no common items**

with

Tj's Write Set.

Diagram

Tj

Write Complete

↓

Ti

Write Starts

Read Set ∩ Write Set = Ø

No conflict occurs.

Condition 3

Both

- Read Set
- Write Set

of Ti

have no common items

with

Write Set of Tj,

and

Tj completes its Read Phase before Ti completes its Read Phase.

Diagram

Read Set ∩ Write Set = Ø

AND

Write Set ∩ Write Set = Ø

Again,

serializability is preserved.

Failure of Validation

If **none** of the three conditions are satisfied,

validation fails.

Therefore,

Transaction

↓

Abort

↓

Restart

Advantages of Validation Technique

- High concurrency.
- No locking.
- No deadlock.
- Lower execution overhead.

- Excellent for read-dominated databases.

Disadvantages

- Repeated transaction restarts.
- Validation may fail frequently under heavy updates.
- Wasted processing time.
- Not suitable for write-intensive applications.

5.6.1 Snapshot Isolation (SI)

Introduction

Snapshot Isolation is another concurrency control technique used in many modern commercial DBMSs.

It combines concepts from

- Multiversion Concurrency Control
- Validation
- Timestamp Ordering

Instead of reading the latest database,

every transaction sees a **snapshot** of the database as it existed when the transaction started.

What is Snapshot?

Definition

A **Snapshot** is the committed state of the database at the moment a transaction begins.

Every transaction works on its own snapshot.

Changes made after the transaction starts are invisible to it.

Working of Snapshot Isolation

Suppose

Transaction T1 starts

at

10:00 AM.

Database

Balance

₹10,000

At

10:05

Another transaction updates

Balance

₹15,000

Transaction T1 still sees

₹10,000

because its snapshot was taken at

10:00.

Features of Snapshot Isolation

- Reads committed data only.
- No dirty reads.
- No non-repeatable reads.
- Phantom records are avoided.
- Each transaction sees a consistent database snapshot.

Temporary Version Store (Tempstore)

Snapshot Isolation maintains old versions in a **temporary version store**.

When a transaction updates data,

the old version is copied to the Tempstore.

Older transactions continue reading the previous version.

New transactions read the latest committed version.

Advantages of Snapshot Isolation

- Very fast read operations.
- No read locks required.
- Better performance than 2PL for read-heavy workloads.
- Reduces blocking between readers and writers.
- Widely used in commercial DBMSs.

Disadvantages

- Rare anomalies affecting serializability may occur.
- Additional storage required for old versions.
- Version management increases complexity.
- Programmers may need to add extra checks if full serializability is required.

Serializable Snapshot Isolation (SSI)

To overcome the rare anomalies of Snapshot Isolation,

an enhanced version called **Serializable Snapshot Isolation (SSI)** is used.

SSI:

- Preserves the benefits of Snapshot Isolation.
- Provides full serializability.
- Eliminates anomalies that can occur under basic SI.
- May incur additional overhead compared to basic SI.

Validation Technique vs Snapshot Isolation

Validation (Optimistic)	**Snapshot Isolation**
Uses validation at transaction end	Uses a database snapshot
No locks during execution	Read operations do not require read locks
Local copies used	Multiple versions maintained

Validation (Optimistic)	Snapshot Isolation
Transactions may restart if validation fails	Older committed versions are read
Best for low-conflict systems	Best for read-intensive workloads

Snapshot Isolation vs Two-Phase Locking

Snapshot Isolation	Two-Phase Locking
No read locks	Read locks required
Readers never block writers	Readers may wait
Uses multiple versions	Single version with locks
Higher read performance	Lower performance for read-heavy systems
Possible rare anomalies	Fully serializable schedules

5.7 Granularity of Data Items and Multiple Granularity Locking

Introduction

In all concurrency control techniques studied so far, such as:

- Two-Phase Locking (2PL)
- Timestamp Ordering (TO)
- Multiversion Concurrency Control (MVCC)
- Validation-Based Concurrency Control

the database is assumed to consist of a collection of **data items**.

A **data item** is the smallest unit on which concurrency control operations such as locking, reading, and writing are performed.

However, one important question arises:

What should be the size of a data item?

Should a lock be applied to:

- One field?
- One record?
- One disk page?
- One file?
- The entire database?

The answer depends on the type of transaction being executed.

The concept that determines the size of a data item is called **Granularity**.

What is Granularity?

Definition

Granularity refers to the **size of the data item** on which a concurrency control mechanism operates.

It determines the level at which locks are applied.

A smaller data item means **fine granularity**, while a larger data item means **coarse granularity**

Types of Data Items

According to the uploaded material, a data item may be one of the following:

- A field value of a database record
- A complete database record
- A disk block (page)
- A complete file
- The entire database

Thus, locking can be performed at different levels depending on the requirements of the transaction.

Granularity Levels

Granularity is broadly classified into two types:

1. Fine Granularity

2. Coarse Granularity

Fine Granularity

Definition

Fine Granularity means locking **small-sized data items**.

Examples:

- Field
- Record

Each record or field receives its own lock.

Example

Suppose a Student table contains

Student 1

Student 2

Student 3

Student 4

Transaction T1 updates Student 1.

Only Student 1 is locked.

Students 2, 3, and 4 remain available for other transactions.

Advantages of Fine Granularity

High Concurrency

Different transactions can access different records simultaneously.

Less Waiting

Transactions rarely block each other.

Efficient Resource Utilization

Only required records are locked.

Disadvantages of Fine Granularity

Large Number of Locks

Every record requires its own lock.

Large Lock Table

The DBMS must maintain many lock entries.

Higher Lock Management Overhead

More lock and unlock operations are required.

Coarse Granularity

Definition

Coarse Granularity means locking **large-sized data items**.

Examples:

- Disk Block
- File
- Entire Database

Example

Suppose

Transaction T1 updates

Student 1

If the file is locked,

Students 2, 3, and 4 are also blocked,

even though they are not being modified.

Advantages of Coarse Granularity

Fewer Locks

Only one lock may protect hundreds of records.

Smaller Lock Table

Less storage required.

Lower Lock Management Overhead

Fewer lock requests are processed

Disadvantages

Poor Concurrency

Many transactions must wait unnecessarily.

Increased Blocking

Even unrelated records become inaccessible.

Comparison: Fine vs Coarse Granularity

Fine Granularity	Coarse Granularity
Small data items	Large data items
More locks	Fewer locks
Higher concurrency	Lower concurrency
Large lock table	Small lock table
High overhead	Low overhead
Better for small transactions	Better for large transactions

Trade-Offs in Choosing Granularity

Choosing the appropriate granularity involves balancing concurrency and overhead.

Large Data Items

When the size of the data item is large:

- Number of locks decreases.
- Lock management becomes easier.

- Storage required for the lock table decreases.
- Concurrency decreases because large portions of the database are locked.

Small Data Items

When the size of the data item is small:

- More locks are required.
- Lock management overhead increases.
- Storage requirements increase.
- Concurrency improves because only the required items are locked.

Example

Suppose Record B and Record C are stored in the same disk block.

Block-Level Lock

Transaction T1 locks Record B.

Since the entire block is locked,

Transaction T2 cannot access Record C,

even though it is different.

Record-Level Lock

Transaction T1 locks Record B.

Transaction T2 can still access Record C.

Therefore,

Record-level locking provides better concurrency

Which Granularity is Best?

There is no single best granularity level.

The choice depends on the type of transaction

Small Transactions

Example

Update one student record.

Best choice

↓

Record-level locking

Large Transactions

Example

Update all student records.

Best choice

↓

File-level locking

Problem with Fixed Granularity

Suppose

Transaction T1 updates an entire file.

Transaction T2 updates only one record.

If record-level locking is always used,

T1 requires thousands of locks.

If file-level locking is always used,

T2 unnecessarily blocks the whole file.

Therefore,

using a fixed granularity level is inefficient.

Solution: Multiple Granularity Locking (MGL)

To overcome this problem,

the DBMS allows locks at different levels.

This technique is called **Multiple Granularity Locking (MGL).**

What is Multiple Granularity Locking?

Definition

Multiple Granularity Locking (MGL) is a locking protocol that allows transactions to acquire locks at **different levels of the granularity hierarchy**.

The lock level can be adjusted dynamically according to the transaction requirements.

Records Records Records Records

Locks can be applied at

- Database
- File
- Page
- Record

Need for Intention Locks

Suppose

Transaction T1 locks an entire file.

Later,

Transaction T2 requests a record lock.

The DBMS must know that a higher-level lock already exists.

Searching every record individually would be extremely slow.

To solve this problem,

Intention Locks are introduced.

What are Intention Locks?

Definition

An **Intention Lock** indicates that a transaction intends to acquire locks on descendant nodes in the hierarchy.

These locks help the lock manager quickly determine whether conflicts exist.

Types of Intention Locks

There are three types.

1. Intention Shared (IS)
2. Intention Exclusive (IX)
3. Shared Intention Exclusive (SIX)

1. Intention Shared (IS)

Definition

An **Intention Shared (IS)** lock indicates that the transaction intends to obtain **Shared Locks** on one or more lower-level nodes.

Example

Transaction wants to read several records.

It first places

IS

↓

Database

↓

File

↓

Page

↓

Shared Lock on Record

2. Intention Exclusive (IX)

Definition

An **Intention Exclusive (IX)** lock indicates that the transaction intends to obtain **Exclusive Locks** on lower-level nodes.

Example

Transaction wants to modify records.

Hierarchy

IX

↓

Database

↓

File

↓

Page

↓

Exclusive Lock on Record

3. Shared Intention Exclusive (SIX)

Definition

A **Shared Intention Exclusive (SIX)** lock means

- Current node is locked in Shared Mode.
- Some descendant nodes will later receive Exclusive Locks.

Example

Suppose a transaction wants

- Read entire file.
- Modify only one record.

Then

File

↓

SIX

↓

Exclusive Lock

↓

Required Record

Lock Modes in MGL

The protocol supports five lock modes.

Lock	Meaning
IS	Intention Shared
IX	Intention Exclusive
S	Shared Lock
SIX	Shared + Intention Exclusive
X	Exclusive Lock

Multiple Granularity Locking Protocol (Rules)

The uploaded material specifies the following rules.

Rule 1

Lock compatibility rules must always be followed.

Conflicting locks cannot be granted simultaneously.

Rule 2

The **root node** (database) must always be locked first.

Rule 3

A node can be locked in **Shared (S)** or **Intention Shared (IS)** mode only if its parent is already locked in **IS** or **IX** mode.

Rule 4

A node can be locked in **Exclusive (X)**, **IX**, or **SIX** mode only if its parent is already locked in **IX** or **SIX** mode.

Rule 5

A transaction cannot acquire new locks after releasing a lock.

This enforces the **Two-Phase Locking Protocol**.

Rule 6

A transaction can unlock a node only if none of its child nodes are currently locked by the same transaction.

Working Example

Suppose

Transaction T1 wants to update

Record R111.

Lock sequence

Database

Advantages of Multiple Granularity Locking

- Supports different transaction sizes.
- Higher concurrency.
- Reduces locking overhead.
- Suitable for mixed workloads.
- Flexible locking mechanism.
- Efficient for both small and large transactions.

Disadvantages

- More complicated implementation.
- Additional intention locks increase complexity.
- Lock compatibility checking becomes more difficult.
- Requires careful management of lock hierarchy.

Granularity Locking vs Multiple Granularity Locking

Granularity Locking	Multiple Granularity Locking
Fixed lock level	Dynamic lock level
Less flexible	Highly flexible
Either record or file locking	Supports all levels
May reduce performance	Better performance
Simple implementation	Complex implementation

Thank You

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and student-focused resources for every semester.

Connect With Us:

www.azdocuments.in [Click Here]

@azdocuments [Click Here]

Join Our Student Community: [Click Here]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.